分 类 号_____ 学号_____
学校代码_____ 密级_____

華中科技大學

硕士学位论文

(学术型□ 专业型☑)

基于驱动程序替换的物联网设备固件仿真技术研究

学位申请人：

学 科 专 业：

指 导 教 师：

答 辩 日 期：

A Dissertation Submitted in Partial Fulfillment of the Requirements for

Research on IoT Device Firmware Emulation Technology Based on Driver Replacement

Candidate :

Major :

Supervisor :

Huazhong University of Science and Technology

Wuhan 430074, P. R. China

April, 2026

独创性声明

本人声明所呈交的学位论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除文中已标明引用的内容外，本论文不包含任何其他人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期：　　年　　月　　日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权华中科技大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本论文属于 保　密 □，在 ____ 年解密后适用本授权书。
不保密 □。

（请在以上方框内打“√”）

学位论文作者签名：　　指导教师签名：

日期：　　年　　月　　日　　日期：　　年　　月　　日

摘　　要

随着物联网技术的快速发展，物联网设备数量呈现爆炸式增长，其固件安全问题日益突出。固件仿真技术是进行固件安全测试和漏洞挖掘的重要手段，然而现有的固件仿真方法存在外设建模成本高、硬件依赖强、自动化程度不足等问题。传统的基于硬件建模的方法需要为每种外设手动建立仿真模型，工作量大且难以覆盖多样化的硬件生态；基于驱动层替换的方法虽然避免了底层硬件建模，但驱动函数的定位与替换仍高度依赖人工干预，难以实现规模化应用。

针对上述问题，本文提出一种结合大语言模型的自动化固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型的代码理解与生成能力自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的稳定运行。

本文的主要工作包括三个方面。首先，设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块，该模块通过静态分析技术，从驱动程序源码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。其次，构建了基于大语言模型的驱动函数分析与替换模块，该模块利用大语言模型理解驱动函数的语义，通过结构化提示词模板和函数分类策略，自动生成可在仿真环境中运行的替代代码，并通过编译修复循环保障代码的可编译性。最后，引入了动态测试反馈闭环机制，当固件在仿真环境中运行出现异常时，系统能够自动分析异常原因并反馈给驱动替换模块，通过”生成—测试—反馈—修正”的闭环流程持续优化仿真质量。

实验结果表明，本文方法能够有效支持多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）的固件仿真，显著降低了固件仿真的技术门槛和人工成本，为漏洞挖掘奠定了基础。

关键词：物联网安全；固件仿真；驱动程序替换；大语言模型；静态分析

Abstract

With the rapid development of Internet of Things (IoT) technology, the number of IoT devices has grown explosively, and firmware security issues have become increasingly prominent. Firmware emulation is an important approach for firmware security testing and vulnerability discovery. However, existing firmware emulation methods face challenges such as high peripheral modeling costs, strong hardware dependencies, and insufficient automation. Traditional hardware modeling-based methods require manually building simulation models for each peripheral, which is labor-intensive and difficult to cover the diverse hardware ecosystem. Although driver layer replacement-based methods avoid low-level hardware modeling, the identification and replacement of driver functions still heavily rely on manual intervention, making it difficult to achieve large-scale application.

To address these issues, this paper proposes an automated firmware emulation method combined with large language models (LLMs). The core idea is to automatically identify hardware-dependent code in firmware through static analysis, use the code understanding and generation capabilities of LLMs to automatically generate alternative driver implementations, and build a dynamic testing feedback loop to iteratively optimize emulation quality, thereby enabling firmware to run stably in emulation environments without manually building peripheral models.

The main contributions of this paper include three aspects. First, a hardware dependency code automatic identification module for embedded firmware is designed and implemented, which automatically extracts hardware-related function call points, peripheral register access patterns, and driver function boundaries from firmware through MMIO pattern recognition, data flow tracking, and recursive induction techniques, constructing a queryable driver knowledge base. Second, a driver function analysis and replacement module based on LLMs is built, which utilizes LLMs to understand driver function semantics, automatically

generates alternative code that can run in emulation environments through structured prompt templates and function classification strategies, and ensures code compilability through a compilation repair loop. Finally, a dynamic testing feedback loop mechanism is introduced. When firmware runs abnormally in the emulation environment, the system can automatically analyze the cause of the abnormality and feed it back to the driver replacement module, continuously optimizing emulation quality through a ”generate-test-feedback-revise” closed-loop process.

Experimental results show that the proposed method can effectively support firmware emulation for multiple embedded platforms (STM32, NXP) and operating systems (FreeRTOS, RT-Thread, bare metal), significantly reducing the technical barrier and manual cost of firmware emulation, laying the foundation for vulnerability discovery.

Keywords: IoT Security; Firmware Emulation; Driver Replacement; Large Language Model; Static Analysis

目　　录

摘　要 I

Abstract II

1　绪论

1.1　研究背景与意义 ..... (1)

1.2　国内外研究现状 ..... (2)

1.3　研究内容与主要贡献 ..... (9)

1.4　论文组织结构 ..... (10)

2　背景知识与相关技术

2.1　物联网设备硬件基础 ..... (13)

2.2　物联网操作系统与驱动 ..... (14)

2.3　大模型代码分析智能体框架 ..... (15)

2.4　静态分析技术 ..... (17)

2.5　固件仿真与模糊测试 ..... (17)

2.6　本章小结 ..... (19)

3　系统设计

3.1　驱动函数接口静态分析与识别模块设计 ..... (21)

3.2　驱动函数分析、替换与编译模块设计 ..... (25)

3.3　动态测试反馈闭环模块设计 ..... (32)

3.4　本章小结 ..... (36)

4　系统实现

4.1　驱动函数接口静态分析与识别模块实现 ..... (37)

4.2　驱动函数分析、替换与编译模块实现 ..... (41)

4.3　动态测试反馈闭环模块实现 ..... (47)

4.4　本章小结 ..... (53)

5 实验设计与结果分析

5.1 实验设计 ..... (55)

5.2 驱动函数识别与替换统计 ..... (57)

5.3 驱动函数分类统计 ..... (59)

5.4 替换功能正确性测试 ..... (60)

5.5 反馈闭环有效性测试 ..... (65)

5.6 方法效率评估 ..... (69)

5.7 模糊测试 ..... (72)

5.8 本章小结 ..... (74)

6 总结与展望

6.1 工作总结 ..... (75)

6.2 论文创新点 ..... (76)

6.3 论文贡献 ..... (77)

6.4 局限性与不足 ..... (77)

6.5 未来工作展望 ..... (79)

参考文献 ..... (81)

A.1 动态反馈闭环替换更新详细统计 ..... (87)

A.2 驱动函数分类详细统计 ..... (87)

1 绪论

1.1 研究背景与意义

近年来，随着物联网（Internet of Things, IoT）技术迅速发展，物联网设备已被广泛用于智能家居、智慧城市、新能源、车联网、医疗健康等多个应用领域。根据 IoT Analytics 发布的《2026 年企业物联网现状报告》，2025 年企业物联网市场同比增长 13%，达到 3240 亿美元。Statista 的数据显示，截至 2024 年，全球物联网设备连接数已超过 170 亿台，预计到 2030 年将增长至 290 亿台以上[1]。物联网操作系统是一种面向物联网设备和应用的轻量级嵌入式操作系统，它可以提供更好的设备管理、数据处理和安全性能。目前，市场上有许多物联网操作系统，例如华为的 OpenHarmony[2]、亚马逊的 FreeRTOS[3]、Linux 基金会的 Zephyr[4] 等。然而，物联网和嵌入式操作系统面临着严峻的安全挑战[5]，如恶意攻击、数据泄露、隐私侵犯等，这些安全问题可能会给用户带来损失和危害。针对物联网恶意软件的安全分析也得到了广泛关注，如 IoTPOT 工具用于分析和检测 IoT 设备中的恶意软件[6]。在真实世界中发生的物联网安全问题也层出不穷。2018 年，一款名为”Mirai” 的恶意软件[7] 利用了 IoT 设备的弱点，通过攻击嵌入式设备的操作系统，将其变成僵尸网络的一部分，对互联网上的服务器发起了大规模的 DDoS 攻击。根据国家互联网应急中心的 CNVD 统计显示,2020 年公开收录的通用型联网智能设备漏洞 3047 个, 新增漏洞数量呈显著增长趋势, 同比增长 28%[8]。这些安全事件的不断发生表明，当前物联网和嵌入式操作系统安全问题的研究刻不容缓。

固件仿真（Firmware Re-Hosting）技术是一种克服固件分析硬件依赖问题的有效方法，它通过在商用计算机和服务器的软件模拟器上运行嵌入式系统固件，实现对嵌入式设备的软件级模拟，使得包含嵌入式系统的固件无需实际硬件即可进行安全测试[9]。然而，当前固件仿真技术的**自动化程度仍然严重不足**，这主要体现在以下几个方面：首先，现有的外设仿真方法大多需要人工编写仿真模型或基于研究者总结的有限的规则自动推断。针对原始硬件特征进行外设建模需要人工分析 MCU 手

册并编写仿真代码，这项工作费时费力，且通用性较差；利用程序分析自动推断部分外设行为的方案虽然可以自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。其次，驱动层替换方案（如 HALucinator[10]）利用了微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过 HAL 函数替换的方式进行固件仿真，但驱动函数的定位、语义理解和替换代码生成仍然需要大量人工干预。分析人员需要手动识别固件中哪些部分存在硬件依赖并需要替换，理解驱动函数与硬件的交互逻辑，并编写相应的替代代码，这个过程不仅耗时耗力，而且容易出错。这种**自动化程度不足导致的人工成本高昂**，严重制约了固件仿真技术在大规模安全测试中的应用，成为当前物联网设备安全领域亟待解决的关键问题。

针对上述问题，本研究致力于提升固件仿真技术的自动化程度，降低人工成本，实现高效的物联网设备固件安全测试。具体而言，本研究提出一种基于驱动程序替换的固件仿真方法，通过静态分析技术自动识别固件中的硬件依赖代码，利用大语言模型的代码理解能力深入分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。该方法的核心创新在于将自动化技术贯穿于驱动函数定位、语义分析和代码生成的全过程，显著减少了对分析人员人工干预的依赖。通过提升固件仿真的自动化水平，本研究不仅能够降低固件安全测试的成本，还能够提高测试效率和覆盖率，为物联网设备的大规模安全测试提供技术支撑，具有重要的理论意义和应用价值。

1.2 国内外研究现状

固件是嵌入式设备中的软件组成部分，通常包含了设备的操作系统、驱动程序、应用程序和配置文件等。固件的安全性对于保护设备和用户数据的完整性和隐私至关重要。然而，由于固件的复杂性、多样性和闭源性，对固件进行有效的安全测试是一项具有挑战性的任务[8]。现有的二进制漏洞检测方法（如 VulHawk[11] 通过跨架构二进制代码搜索检测已知漏洞）虽然取得了一定成效，但仍难以全面覆盖固件中深层次的安全问题。

基于固件仿真的模糊测试技术在近年来受到了越来越多的关注和研究，出现了

一些代表性的工具和系统，如 Firmadyne[12]、Fuzzware[13] 等。这些工具和系统在不同程度上实现了对固件仿真和模糊测试的支持，并在实际的固件中发现了一些新的漏洞。然而，基于固件仿真的模糊测试技术仍然存在一些挑战和问题，如仿真环境的准确性、测试用例的有效性等，需要进一步的改进和优化。下面将从固件仿真技术、嵌入式设备模糊测试技术两个方面阐述当前基于固件仿真的嵌入式设备固件模糊测试技术研究现状。

1.2.1 固件仿真技术研究现状

当前固件仿真的主要挑战是为模拟器中没有实现的硬件提供有效的输入。诸如 QEMU[14] 之类的仿真器提供了模拟各种 CPU 体系结构的能力，但对商用嵌入式系统中使用的各种各样的外设，例如计时器、UART、以太网控制器等，提供的支持却十分有限，而这些外设对于正确实现固件仿真却是十分必要的。

固件仿真技术的发展经历了从依赖真实硬件到全自动化模拟的演进过程。早期研究采用硬件在环（Hardware in the Loop, HIL）的方法，如 Zaddich 等人提出的 Avatar[15] 和 Muench 等人提出的 Avatar2[16]，通过混合执行的方式将固件分析过程中对外设的访问转发到物理硬件中。然而，这种方案仍然依赖实际设备，且仿真环境和物理设备之间的切换影响了运行速度。为摆脱对真实硬件的依赖，研究者提出了多种自动化程度较高的仿真方案，主要包括以下三类。

1.2.1.1 基于原始硬件的外设建模方案

解决外设依赖的一种方法是在底层内存映射寄存器接口上实现硬件，或通过观察真实硬件的行为来构建外设模型。然而这是一项费力的任务，扩展性不好。当前主流的仿真器 QEMU 也仅能仿真 10 余种设备，远不能满足固件仿真工作的需求。

Gustafson 等人提出的 Pretender[17] 工具通过观察固件与物理设备之间真实发生的交互记录，使用机器学习、模式识别等方法为每个外设构建模型，再结合 QEMU 或 angr[18] 等仿真引擎实现固件仿真。这种方法需要真实设备的支持，但在观测到足够的交互模式后，可以自动生成较为准确的外设模型。Zhou 等人提出的 sEmu[19] 工具通过 NLP（自然语言处理）技术，从 MCU 手册中提取外设的寄存器信息，构建

外设模型，再结合 QEMU 或 angr 等仿真引擎实现固件仿真。这些方案的优点是能够对真实设备的行为进行精确模拟，提高测试效率和覆盖率，但缺点是依赖真实设备观测，通用性差。

1.2.1.2 基于程序分析的外设建模方案

基于程序分析的外设建模方案通过对固件代码进行静态或动态分析，提取固件与外设之间的交互逻辑，构建虚拟外设模型以替代真实硬件。

Feng 等人提出的 P2IM[20] 工具通过动态符号执行和模糊测试，自动识别固件中的输入点和输出点，构建与真实设备相似的虚拟模型。Cao 等人提出的 Laelaps[21] 在因访问未实现的外设而卡住时，切换到符号执行以找到正确输入，引导 QEMU 找到与实际执行最相近的路径。Zhou 等人提出的 uEmu[22] 工具利用符号执行分析固件镜像，将未知的外设寄存器表示为符号，构建推断规则并存储在知识库中。Scharnowski 等人提出的 Fuzzware[13] 使用精确的内存映射输入输出（Memory-Mapped I/O，MMIO）建模实现有效的固件模糊测试，通过确定固件逻辑如何实际使用硬件生成的值，自动生成模型以提高模糊测试输入数据的有效性。Shoshitaishvili 等人提出的 Firmalice[23] 通过符号执行自动检测二进制固件中的身份验证绕过漏洞。Johnson 等人提出的 Jetset[24] 利用符号执行推断固件期望的外设行为，自动生成外设模型以实现固件 rehosting。Mera 等人提出的 DICE[25] 针对 DMA 输入通道的自动仿真问题，为固件动态分析提供硬件无关的 DMA 输入通道模拟支持。这些基于程序分析的外设建模方案实现了一定程度的自动化，但这些方案难以完全覆盖复杂外设的所有行为，可能存在误报或漏报。

1.2.1.3 基于系统或驱动层替换的外设仿真方案

基于系统或驱动层替换的仿真技术，又称高级仿真（High Level Emulation, HLE），利用固件中的公共抽象来消除为外设提供低级支持的需要。Chen 等人提出了一个针对 Linux 内核的物联网设备固件仿真平台 Firmadyne[12]，通过修改内核和驱动来添加硬件支持，实现物联网设备的全系统仿真。该工具解决了定制化外围 I/O、非易失

性内存、动态文件等仿真技术难题。Li 等人创建了 Para-rehosting[26] 工具，该工具的主要思想是利用 POSIX 接口抽象并实现了一个便携式 MCU，以将嵌入式系统源代码重新编译到商用机器的本机可执行文件中，以便利用 x86 上许多现有的复杂软件测试工具对代码进行测试。Clements 等人提出了 HALucinator[10] 工具，该工具利用了微控制器制造商提供的硬件抽象层（Hardware Abstraction Layer, HAL），通过对硬件抽象层函数的抽象与替换进行固件仿真。Seidel 等人提出的 SAFIREFUZZ[27] 在 HALucinator 的基础上进一步优化了 HLE 方案的模糊测试性能，通过近原生 rehosting 将 ARM Cortex-M 固件作为 Linux 用户态进程运行，结合动态二进制改写技术降低了仿真开销，相比 HALucinator 实现了平均 690 倍的吞吐量提升。

然而，上述基于系统或驱动层替换的仿真技术自动化程度较差，难以适应大规模安全测试的需要。具体而言，这类技术面临以下两个核心难点：一是难以自动化定位需要进行替换的驱动程序。在嵌入式程序中，识别哪些部分存在硬件依赖并需要替换，往往依赖分析人员的手动干预。手动分析嵌入式程序的复杂性使得这一过程既耗时又容易出错。二是难以自动化生成替换程序。除了识别硬件相关功能，现有的方案也缺乏自动化生成替代程序的能力。分析人员必须手动分析函数的功能，理解其与硬件和中间件的交互，并编写相应的替代代码，这个过程不仅繁琐，而且容易出错，进一步增加了人工成本。

1.2.1.4 固件仿真技术对比

表 1.1 对比了当前主流固件仿真技术的优缺点。

在这三类方案中，基于系统或驱动层替换的固件仿真方法从固件系统层和驱动函数入手，较好地解决了基于程序分析和原始硬件建模的方法对复杂外设难以完全模拟或建模工作量大等痛点。然而，现有驱动层替换方案在自动化方面仍存在明显不足：一方面，驱动函数的定位依赖分析人员的手动干预，识别固件中哪些部分存在硬件依赖并需要替换是一个耗时且容易出错的过程；另一方面，替换程序的生成也缺乏自动化支持，分析人员需要手动分析函数功能、理解其与硬件的交互，并编写相应的替代代码。为解决上述问题，需要设计一种能够自动化定位需要替换的驱

表 1.1 当前主流固件仿真技术对比

方案	典型工具	优点	缺点
基于原始硬件的外设建模	QEMU, Pretender	可对外设行为进行精确模拟，提高测试效率和覆盖率	需要为每种外设专门建模，工作量大，通用性差，依赖真实设备观测
基于程序分析的外设建模	P2IM, uEmu, Laelaps, Fuzzware	可根据程序自身信息自动推断外设行为，减少人工干预，提高通用性	难以完全覆盖复杂外设的所有行为，可能存在误报或漏报
基于系统或驱动层替换	HALucinator, SAFIREFUZZ, Firmadyne, Para-rehosting	在系统层或驱动层对函数进行抽象和替换，降低对硬件的依赖，提高测试速度和灵活性	驱动函数的定位与替换依赖人工干预，自动化程度不足，难以适应大规模安全测试

动函数、并利用大模型或代码分析技术自动生成替代程序的方案，以提升固件仿真过程的自动化程度，实现高效的物联网设备程序仿真与动态测试。

1.2.2 嵌入式系统模糊测试技术发展现状

嵌入式系统模糊测试技术是一种针对嵌入式设备的软件安全测试方法，它可以自动或半自动地生成随机或变异的输入数据，发送给嵌入式设备的 Web 服务或其他接口，监控设备的运行状态，发现潜在的漏洞或缺陷。目前，嵌入式系统模糊测试技术主要以下两个方向：面向真实设备的黑盒模糊测试和基于固件仿真的灰盒模糊测试。

1.2.2.1 面向真实设备的黑盒模糊测试

直接对嵌入式设备进行模糊测试，不需要对设备进行任何修改或插桩，只需要获取设备的网络接口或串口等信息，然后使用模糊测试工具生成并发送测试用例，监控设备的响应和异常。Chen 等人开发的 IoTFuzzer[28] 工具，就利用了应用程序与物联网设备的交互信息，通过网络接口向嵌入式设备发送模糊测试数据，进行模糊测试工作。Feng 等人开发的 Snipuzz[29] 通过分析设备的响应消息，推断消息片段（snippets），基于消息片段进行变异以提升变异质量。Liu 等人提出的 Labrador[30] 利

用设备响应信息引导模糊测试方向，实现了面向黑盒 IoT 设备的高效定向模糊测试。这种方法的优点是简单易用，不依赖于设备的内部结构或代码，适用于黑盒测试场景。缺点是测试效率较低，覆盖率较差，难以发现深层次的漏洞。

1.2.2.2 基于固件仿真的灰盒模糊测试

基于固件仿真的模糊测试，通过对嵌入式设备的固件进行逆向工程，提取出可执行的二进制代码，然后使用仿真器在虚拟环境中运行固件，并对其进行模糊测试。基于该方案的仿真工具有 Fuzzware[13]、hoedur[31]、FirmAFL[32]、SFuzz[33]、SHiFT[34]等。其中，SFuzz 通过基于切片的模糊测试方法解决 RTOS 单体二进制中任务与服务耦合导致的测试困难，SHiFT 提出半托管式模糊测试方法，通过将嵌入式应用的部分执行委托给宿主机来提升测试效率。为了进一步提升模糊测试的效率和覆盖率，研究者提出了多种混合模糊测试方法，如 Driller[35] 通过选择性符号执行增强模糊测试，QSYM[36] 提供了实用的混合执行引擎用于混合模糊测试，Angora[37] 通过原则性搜索实现高效模糊测试。这种方法的优点是可以对固件进行插桩或修改，获取更多的运行信息，提高测试效率和覆盖率，发现更多的漏洞。此外，Shi 等人提出的 IPEA[38] 采用解耦架构将资源密集的分析任务从微控制器卸载到工作站，仅保留轻量级探针在固件中采集执行信息，实现了非侵入式的真机固件测试。缺点是需要对固件进行复杂的逆向分析，可能遇到加密或保护机制，同时仿真器可能存在不完全或不准确的问题，导致测试结果与真实设备不一致。

1.2.3 大模型程序分析与代码生成技术研究现状

大语言模型（Large Language Model, LLM）在代码理解、生成和程序分析领域取得了突破性进展。以 OpenAI 的 GPT 系列、Anthropic 的 Claude 3.5[39]、Google 的 Gemini 3[40]、DeepSeek-R1[41]、智谱 GLM-5[42] 和 MiniMax M2.7[43] 为代表的大语言模型在代码能力方面展现出卓越的性能。这些模型不仅能够理解和生成高质量的代码，还能够进行复杂的推理、调试和优化任务。这些进展为驱动函数分析和驱动代码生成的自动化提供了新的方向。

近年来，研究者开始探索将 LLM 应用于程序分析和代码生成领域，涌现出多种

增强策略以提升模型在复杂编程任务中的表现。早期 LLM 通过零样本提示能够实现基本的代码生成任务，但在处理复杂编程任务时该方案的准确性和可靠性仍存在不足。为克服这些局限，研究者发展了以下增强策略：

指令微调策略：通过对预训练 LLM 在特定领域数据集上进行精细调整，显著提升模型对特定编程任务的理解和执行能力[44]。DolphCoder 模型通过多样化的指令和自评估机制，结合代码评估目标有效提升了代码生成能力[45]。CodeLSI 框架则通过低秩优化与领域特定指令微调相结合，提高了代码生成的领域特异性和成本效益[46]。

测试驱动迭策略：利用测试用例作为反馈信号指导模型逐步修正生成的代码，直至通过所有测试。这种方法通过隐式执行监督信号，结合搜索、重排序或迭代修正等机制动态优化模型输出[47]。RLCF（Reinforcement Learning with Compiler Feedback）方法通过编译器反馈对 LLM 进行额外训练，解决生成代码中可能存在的语法违规问题[48]。CodeDPO 框架则将偏好学习整合到代码生成过程中，优化模型使其优先选择正确且高效的解决方案[49]。

多步推理策略：思维链（Chain-of-Thought, CoT）和程序思维（Program-of-Thought, PoT）等方法通过将复杂问题分解为多个可管理的子步骤，提升模型解决复杂问题的能力[50]。MoTCoder 引入模块化的思维指令框架来应对复杂编程任务[51]。这种程序思维方法则将中间推理步骤编译为可执行代码片段，实现了”推理即执行”，大幅提升了数值计算和 API 调用类任务的准确性。

工具集成策略：通过集成编译器、调试器、静态分析工具等外部工具，扩展模型的实际应用能力。这种范式将 LLM 定位为”AI 协调者”，通过标准化工具调用协议调度各种工具，形成感知-决策-执行的闭环[52]。MCCoder 系统专注于生成运动控制代码，并结合严格的验证机制以确保代码安全性[53]。RGD 代理调试器利用多 LLM 通过细化和生成指导来自动修复代码中的错误[54]。Agents4PLC 框架通过基于 LLM 的智能体自动化了可编程逻辑控制器（PLC）代码生成与验证，构建了端到端的”分析 → 生成 → 符号验证 → 反馈精炼”流水线[55]。其中形式化规约被用作生成约束，而验证失败轨迹则被结构化为自然语言反馈以驱动下一轮生成，提高了工业控制系统的操作效率和安全性。

1.2.4 现有研究的局限性分析

综合上述研究现状，现有固件仿真技术主要存在以下局限性：

（1）外设建模方案局限性强，推断规则有限难以覆盖复杂外设状态机逻辑

现有基于程序分析的外设建模方案虽然能够自动推断部分外设行为，但推断规则有限，难以覆盖复杂外设的状态机逻辑。网络控制器、USB 设备等复杂外设具有多通道、DMA 传输、中断嵌套等特性，现有自动推断方法往往只能处理简单的 MMIO 寄存器读写，无法准确模拟这些复杂交互行为。硬件在环方案和基于原始硬件的建模方案虽然可以处理复杂外设，但需要为每种外设专门配置物理设备连接或人工分析数据手册并编写仿真代码，工作量巨大且通用性差。

（2）驱动仿真人工依赖大，无法适应大规模需要

现有驱动层替换方案（如 HALucinator）的自动化程度不足，驱动函数的定位与替换高度依赖人工干预。分析人员需要手动识别固件中哪些部分存在硬件依赖并需要替换，理解驱动函数与硬件的交互逻辑，并编写相应的替代代码。这个过程不仅耗时耗力，而且容易出错，导致仿真质量高度依赖分析人员的经验。这种对人工的依赖严重制约了固件仿真技术在大规模安全测试中的应用，难以适应物联网设备固件数量庞大、种类繁多的特点。

（3）缺乏自动化反馈机制，难以迭代优化仿真质量

当前大多数仿真工具采用”一次性配置”的模式：分析人员手动配置仿真环境后运行测试，如果发现问题则需要人工介入调试和修复。这种模式缺乏自动化的反馈闭环机制——当仿真过程中出现异常（如程序卡死、断言失败、外设访问超时）时，系统无法自动分析原因并调整仿真策略。这导致仿真质量的提升高度依赖分析人员的经验，难以适应大规模、多样化的固件测试需求。

1.3 研究内容与主要贡献

为解决上述问题，本文提出一种基于驱动程序替换的物联网设备固件仿真方法。该方法通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型分析驱动函数语义并自动生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量。

本文主要的创新点和贡献有三个：

（1）本研究设计并实现了一个面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别、数据流追踪等技术，从固件二进制代码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界，构建可查询的驱动知识库。与现有依赖人工标注或有限规则推断的方法相比，该模块能够自动处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，显著降低了驱动函数定位的人工成本。

（2）本研究设计并实现了一个基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，本模块能够处理不同类型的驱动函数（如初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。与现有基于预设返回值的简单钩子函数（Hook）方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

（3）本研究设计并实现了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种”生成-测试-反馈-修正”的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

1.4 论文组织结构

本文共分为六章，从绪论、背景知识、系统设计、系统实现、实验验证到总结展望，逐步深入地阐述基于驱动程序替换的物联网设备固件仿真方法。图 1.1 展示了本文的整体组织结构。

第一章：绪论。阐述物联网设备安全问题的研究背景，分析固件仿真技术在物联网安全测试中的作用。梳理国内外在固件仿真、嵌入式系统模糊测试、大模型程

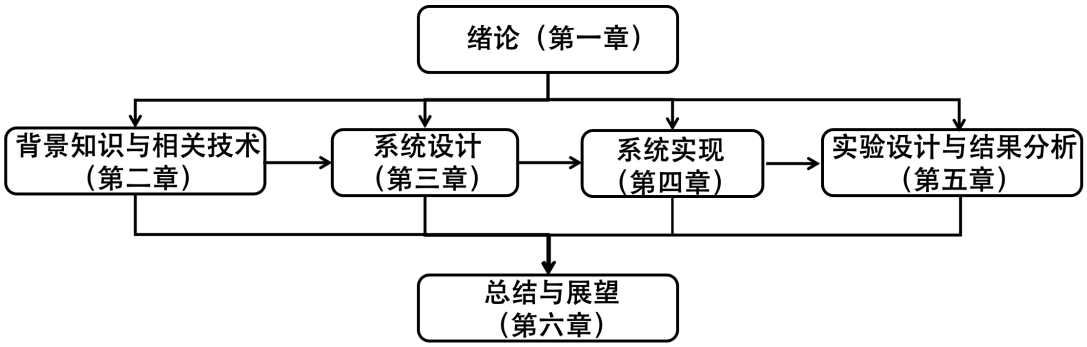

图 1.1 文章架构图

序分析与代码生成等领域的研究现状，分析现有技术存在的局限性，明确本文的研究内容和主要贡献。

第二章：背景知识与相关技术。介绍物联网设备固件仿真研究涉及的基础知识和技术工具。首先阐述 ARM Cortex-M 架构特性和内存映射 I/O 机制，然后介绍实时操作系统与硬件抽象层的概念，接着讲解大语言模型代码分析的智能体框架设计模式，最后介绍 CodeQL 静态分析平台和 QEMU 仿真器与 AFL 模糊测试工具的原理与应用。

第三章：系统设计。阐述基于驱动程序替换的固件仿真方法的整体架构设计。介绍系统的三大核心模块：驱动函数接口静态分析与识别模块负责定位硬件相关代码并构建可查询知识库，驱动函数分析替换与编译模块负责理解驱动函数语义并生成替代实现，动态测试反馈闭环模块负责在仿真环境中运行固件并采集反馈信息。详细说明各模块的职责划分、接口设计和协作机制。

第四章：系统实现。详细阐述各关键模块的具体实现技术。包括 CodeQL 查询规则的设计与实现，用于识别 MMIO 访问模式和追踪数据流；LangGraph 智能体工作流的构建，用于驱动函数分类和替换代码生成；多种替换策略的实现，用于处理不同类型的驱动函数；以及基于 Unicorn 的仿真平台搭建和反馈闭环的实现细节。

第五章：实验设计与结果分析。通过实验验证本文方法的有效性和实用性。首先介绍实验环境的软硬件配置和测试用例的构建，之后从功能测试与性能测试两个方面，多个维度评估系统能力，包括静态分析模块的识别准确率、固件在仿真环境

中的运行正确性、与现有方法的对比分析以及在实际固件上的应用验证。

第六章：总结与展望。总结全文的研究工作，概括本文在硬件依赖代码自动识别、基于大语言模型的驱动函数替换、动态测试反馈闭环等方面的创新点和贡献。客观分析当前方法在源码依赖、LLM 成本、平台覆盖和复杂外设模拟等方面存在的局限性，并对支持无源码固件、扩展平台支持、提升模拟精度和安全性验证等未来研究方向进行展望。

2 背景知识与相关技术

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础、物联网操作系统与驱动框架、大模型代码分析技术以及固件仿真与模糊测试技术，为后续章节的论述奠定基础。

2.1 物联网设备硬件基础

物联网设备通常采用嵌入式微控制器（Microcontroller Unit, MCU）作为核心处理器。常见的 MCU 架构包括 ARM Cortex-M 系列、RISC-V 等。这些处理器具有低功耗、低成本、实时性强等特点，广泛应用于物联网领域。本节重点介绍 ARM Cortex-M 架构的特性以及内存映射 I/O 机制。

2.1.1 ARM Cortex-M 架构特性

ARM Cortex-M 是 ARM 架构处理器核心中低端系列的统称，专为低成本、高能效的微控制器设计，广泛应用于物联网设备。该系列采用 Thumb-2 指令集，在提供 32 位性能的同时保持 16 位代码密度。Cortex-M 架构提供了多项增强实时性和可靠性的特性，包括嵌套向量中断控制器（NVIC）用于中断管理、系统定时器用于提供可编程的周期性中断并支持操作系统的任务调度、以及主栈与进程栈的栈模式切换机制以实现多任务并发和内存保护。

2.1.2 内存映射 I/O

内存映射 I/O（Memory-Mapped I/O, MMIO）是嵌入式系统中外设与处理器通信的主要方式。MMIO 将外设寄存器映射到处理器的统一内存地址空间中，处理器通过普通的内存读写指令（如 LDR/STR）访问特定内存地址，即可完成对外设的控制和数据传输。这种设计简化了编程模型，使得外设访问与内存访问具有相同的语法形式。

ARM Cortex-M 架构采用统一的 32 位地址空间，按功能划分为代码区（Flash）、SRAM 区、外设区和系统区。外设区是 MMIO 的核心区域，所有片上外设的寄存器

都被映射到此区域，不同厂商的外设被分配到不同的地址段。系统区位于地址空间高端，包含 NVIC、SysTick、MPU 等系统级组件的控制寄存器。理解 MMIO 地址空间布局对于固件仿真至关重要，仿真器需要针对外设区和系统区实现专门的访问处理逻辑。

外设寄存器按照功能可分为控制寄存器、状态寄存器和数据寄存器三类。控制寄存器用于配置外设的工作模式和参数，如 USART 的 CR1 可设置波特率、数据位长度等。状态寄存器反映外设的当前工作状态，如 USART 的 SR 包含发送缓冲区空（TXE）、接收缓冲区非空（RXNE）等标志位，其值由硬件自动更新。数据寄存器用于数据传输，如 USART 的 DR 用于发送和接收数据，访问数据寄存器通常会触发外设执行相应操作。

2.2 物联网操作系统与驱动

2.2.1 实时操作系统概述

实时操作系统（Real-Time Operating System, RTOS）是物联网设备中广泛采用的管理任务调度和资源分配的软件系统。RTOS 能够及时响应外部事件或数据，并在规定的时间内完成任务，其工作原理是通过一个实时任务调度器来管理系统资源，按照优先级或者时间片来分配 CPU 时间给不同的任务，保证重要或者紧急的任务优先执行。

物联网领域常见的 RTOS 包括 FreeRTOS[3]、Zephyr[4]、RT-Thread[56] 等。FreeRTOS 是 Amazon 开源的实时操作系统，专为微控制器和小型微处理器设计，内核精简，最小可运行在不到 10KB 的 RAM 上，是物联网领域使用最广泛的 RTOS 之一。Zephyr 是 Linux 基金会管理的开源实时操作系统，支持多种处理器架构，提供更完整的操作系统功能，包括文件系统、网络协议栈、设备驱动模型等，采用模块化设计以平衡功能丰富性和资源占用。RT-Thread 是中国开源社区主导开发的实时操作系统，采用微内核架构，并提供了丰富的组件生态。

2.2.2 硬件抽象层

硬件抽象层（Hardware Abstraction Layer, HAL）是嵌入式软件架构中的关键组成部分，它位于硬件驱动层和上层应用之间，为应用程序提供统一的硬件访问接口。HAL 层的核心目标是屏蔽底层硬件的差异性，使得上层应用代码可以在不同的硬件平台之间移植，而无需修改业务逻辑。

HAL 层的接口设计遵循统一的命名和功能规范。以 STM32 HAL 库为例，每个外设类型提供初始化函数（如 `HAL_UART_Init`）、数据传输函数（如 `HAL_UART_Transmit`）、状态查询函数（如 `HAL_UART_GetState`）和中断管理函数（如 `HAL_UART_Receive_IT`）。由于 HAL 函数封装了底层硬件操作细节，仿真器可通过替换 HAL 层函数来实现硬件行为模拟，无需深入理解寄存器语义，相比寄存器级模拟更加高效且易于实现。

2.3 大模型代码分析智能体框架

大语言模型（Large Language Model, LLM）在代码理解和生成方面展现出强大的能力，为自动化程序分析提供了新的技术途径。本节重点阐述代码分析工作流范式和基于 LLM 的智能体开发框架，为后续章节的驱动函数分析和替换工作奠定技术背景。

2.3.1 代码分析工作流设计模式

基于 LLM 智能体框架构建代码分析系统时，研究者提出了多种工作流设计模式。这些模式为处理代码分析任务中常见的逻辑推理、工具调用、多步骤协作等挑战提供了系统化思路。

ReAct 模式：ReAct（Reasoning + Acting）将推理与行动交替进行[57]。在代码分析中，LLM 可先推理分析目标（如”需要理解该函数的调用关系”），再调用静态分析工具获取信息，观察结果后继续下一步推理。该模式使 LLM 能够动态收集信息并逐步完成复杂分析。

规划-执行模式：该模式将任务分为规划与执行两阶段。规划阶段生成步骤化的执行计划，执行阶段按序完成。其优势在于结构清晰、便于调试和过程监控。

多智能体协作模式：该模式使用多个专门化的智能体协同完成任务[58]。例如在驱动函数替换中，可设计分析智能体、生成智能体、验证智能体与协调智能体，通过消息传递或共享状态完成复杂代码分析与生成任务。本研究在驱动函数替换模块中，综合运用了 ReAct 模式与多智能体协作模式，以应对硬件相关代码替换的复杂性与正确性要求。

2.3.2 LLM 智能体开发框架

为了更好地利用大语言模型构建复杂的应用系统，研究人员开发了多种智能体（Agent）框架。这些框架提供了结构化的方式来组织 LLM 的调用、管理状态、集成外部工具，使开发者能够构建可靠的、可扩展的 LLM 应用。

2.3.2.1 LangChain 框架

LangChain 是目前最流行的 LLM 应用开发框架之一，提供了模型、提示模板、链、代理、工具和记忆等核心组件。模型组件封装了与不同 LLM 提供商的交互接口，便于切换和比较。提示模板允许开发者定义带变量的提示词，在运行时动态填充，提高复用性。链将多个组件按顺序组合，构建复杂的处理流程。代理是更高级的抽象，允许 LLM 根据用户输入自主选择合适的工具（如搜索引擎、代码执行器）并决定后续行动。工具是代理可以调用的外部功能，框架提供了大量内置工具，开发者也可自定义。记忆组件维持应用在多次调用之间的状态，例如在多轮对话中存储历史对话。

2.3.2.2 LangGraph 状态图

LangGraph 是 LangChain 生态中用于构建有状态、可循环的智能体工作流的库。与 LangChain 的链式结构不同，LangGraph 基于状态图的概念，状态在各个节点之间传递和更新。节点是处理单元，边定义转换关系并支持条件分支和循环，适用于迭代优化等场景。LangGraph 还支持检查点机制，可将工作流状态持久化存储，支持长时间运行任务的暂停和恢复。

2.4 静态分析技术

静态分析技术是软件工程和安全研究的重要基础，旨在通过在不执行程序的情况下，通过分析源代码或二进制代码来推断程序的属性。静态分析的优势在于能够覆盖程序的所有可能执行路径，不受测试用例的限制。常见的静态分析技术包括数据流分析、控制流分析、指针分析、抽象解释等。

2.4.1 CodeQL 代码分析平台

CodeQL 是 GitHub 开发的语义级代码分析平台，支持多种编程语言（包括 C/C++、Java、Python、JavaScript 等）的安全漏洞检测和代码质量分析。CodeQL 的核心思想是将源代码转换为可查询的数据库，然后通过声明式查询语言（QL）进行灵活的分析。与传统基于规则的静态分析工具不同，CodeQL 提供了表达力丰富的查询语言，研究人员可以通过编写查询来查找特定的代码模式、追踪数据流、分析控制依赖等复杂问题。

CodeQL 的工作原理分为两个阶段。第一阶段是数据库构建，CodeQL 对源代码进行解析，生成抽象语法树（AST）、控制流图（CFG）、数据流图（DFG）等多层次的结构化表示，并将这些信息存储在数据库中。第二阶段是查询执行，研究人员使用 QL 语言编写查询，在数据库上执行，获取分析结果。CodeQL 的查询语言基于 Datalog，采用谓词逻辑的表达方式，能够自然地表达”所有存在某种模式的代码片段”或”从源点到汇点的数据流路径”等复杂查询。

CodeQL 在程序分析领域应用广泛，包括安全漏洞检测（如 SQL 注入、缓冲区溢出）、代码质量分析（如死代码检测、复杂度分析）、以及依赖关系分析（如函数调用图、数据流追踪）。本研究利用 CodeQL 的数据流追踪能力，建立 MMIO 访问与函数参数之间的关联关系，为驱动函数识别和语义分析提供支撑。

2.5 固件仿真与模糊测试

固件仿真是物联网安全研究的重要技术手段。由于物联网设备的硬件多样性和封闭性，直接在真实设备上进行安全测试面临诸多困难。固件仿真技术通过软件模拟硬件环境，使研究人员能够在通用计算平台上运行和分析嵌入式固件，从而降低

测试成本并提高效率。模糊测试作为一种高效的漏洞挖掘技术，与固件仿真相结合，能够在仿真环境中自动化地发现固件中的安全漏洞。

2.5.1 QEMU 仿真器

QEMU[14] 是目前应用最广泛的开源机器仿真器，支持多种处理器架构和外设仿真，其设计目标是提供高性能、高可移植性的系统仿真能力，使其成为固件仿真研究的重要基础平台。QEMU 支持两种主要的仿真模式：用户模式仿真（User Mode Emulation）允许在一种架构上运行为另一种架构编译的单个程序，适用于分析用户空间程序；系统模式仿真（System Mode Emulation）提供完整的硬件系统仿真，包括处理器、内存、中断控制器、外设等组件，可以运行完整的操作系统或裸机固件，这是固件仿真的主要工作模式。

QEMU 的核心是动态二进制翻译引擎（Tiny Code Generator, TCG），它将目标架构的指令块翻译为中间表示（IR），然后再将 IR 翻译为宿主机指令，这种两级翻译机制使 QEMU 能够支持多种目标架构和宿主架构的组合。为了提高性能，TCG 采用即时编译（JIT）技术，将翻译结果缓存在内存中供后续执行重复使用。

Unicorn[59] 是基于 QEMU 的 TCG 引擎开发的轻量级 CPU 模拟器，专门针对安全研究和逆向工程场景优化。与 QEMU 提供完整系统级仿真的设计不同，Unicorn 去除了 QEMU 的系统级组件和外设仿真，只保留指令集模拟核心，因此具有更好的灵活性和可控性。Unicorn 提供了丰富的钩子（Hook）机制，允许用户在指令执行、内存访问、中断触发等事件上注册自定义回调函数，从而在不修改模拟器源码的情况下实现对执行过程的细粒度监控和干预。此外，Unicorn 以库的形式提供编程接口（支持 Python、C 等多种语言），便于嵌入到其他工具链中。本研究选用 Unicorn 作为固件动态测试的仿真引擎，利用其 ARM Thumb 模式模拟 ARM Cortex-M 系列微控制器，并通过钩子机制实现驱动函数拦截、运行日志采集和仿真数据注入等功能。

2.5.2 模糊测试技术

模糊测试（Fuzzing）是一种自动化软件测试技术，通过向目标程序输入大量随机、半随机或结构化的测试用例，来发现程序中的安全漏洞和缺陷[60]。模糊测试的

基本工作流程包括：生成或变异测试用例、将测试用例输入目标程序、监控程序执行状态（如崩溃、断言失败、内存错误等）、记录和分析异常。

American Fuzzy Lop（AFL）[61] 是最具影响力的覆盖率引导模糊测试工具之一。AFL 的核心思想是通过代码插桩技术记录程序执行过程中的覆盖率信息，利用这些信息指导测试用例的变异和选择，优先保留能够触发新执行路径的测试用例，从而在有限的测试时间内探索更多的程序状态，提高漏洞发现的概率。AFL 的工作流程包含两个核心环节：覆盖率记录和变异策略。在覆盖率记录环节，AFL 通过编译时插桩或运行时插桩（利用 QEMU 的动态翻译功能）在目标程序中插入覆盖率记录代码，记录程序执行过程中经过的基本块和边。在变异策略环节，AFL 维护一个测试用例队列，不断从队列中选择测试用例，对其进行变异，执行变异后的测试用例，并根据覆盖率变化决定是否将新用例加入队列。AFL 的变异策略包括位翻转、字节删除、字节插入、块拼接、算术运算等多种操作，能够产生多样化的测试输入。通过这种迭代式的测试和筛选机制，AFL 能够逐步探索程序的各种执行路径，自动发现潜在的漏洞和缺陷。

在嵌入式固件模糊测试领域，AFL 常与仿真器（如 QEMU）结合使用。研究者在 QEMU 的动态翻译机制基础上实现插桩，使 AFL 能够对在仿真环境中运行的固件进行模糊测试。这种方法不需要修改固件源码，能够直接对二进制固件进行测试，显著提高了测试的自动化程度和可扩展性。

2.6 本章小结

本章介绍了物联网设备固件仿真研究涉及的基础知识，包括物联网设备硬件基础、物联网操作系统与驱动、大模型代码分析智能体框架、静态分析技术、固件仿真与模糊测试技术，为后续章节的论述奠定基础。

3 系统设计

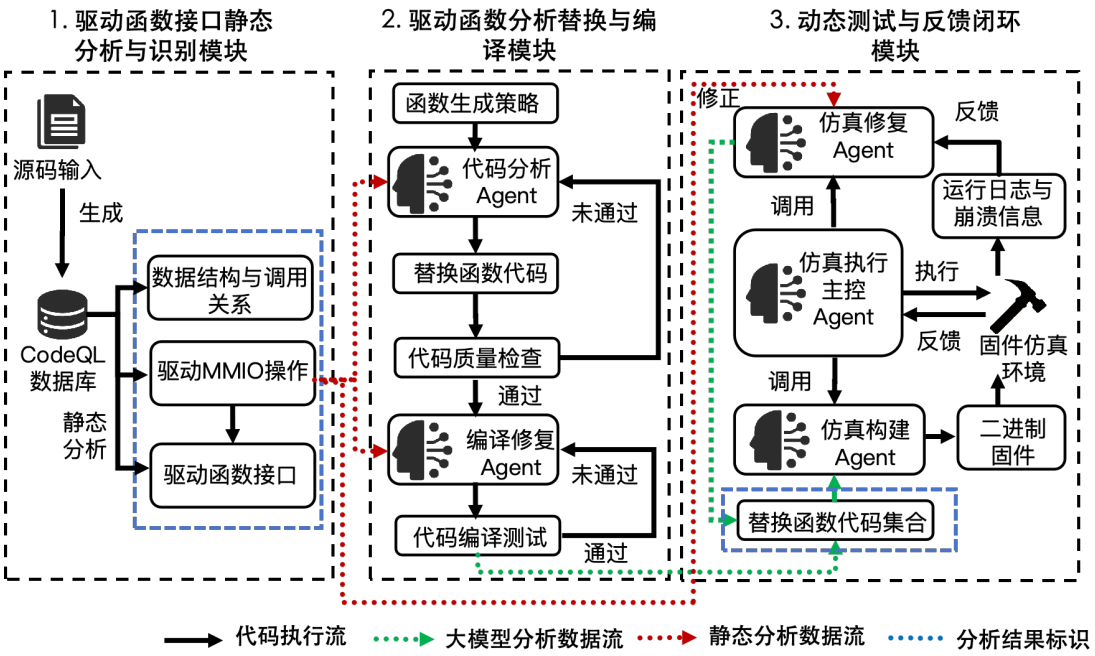

图 3.1 系统整体架构图

本研究设计并实现了基于驱动程序替换的固件仿真系统，该系统分为三个模块：驱动函数接口静态分析与识别模块、驱动函数分析、替换与编译模块、动态测试反馈闭环模块。整体方案架构图如图 3.1所示。其中，驱动函数接口静态分析与识别模块负责对固件源码进行静态分析，定位硬件相关代码，抽取驱动函数、MMIO 访问点、依赖数据结构、调用关系等信息，并以统一的数据模型对外提供查询接口。驱动函数分析、替换与编译模块根据静态分析模块输出的驱动分析结果，使用大语言模型对驱动函数进行类型分类并采用合适的函数替换策略，生成可替代实现，同时将替代实现集成回工程并驱动构建。动态测试反馈闭环模块负责在仿真环境中运行固件，采集运行日志、崩溃信息等，并将错误信息结构化，触发重新分析与再生成。

驱动函数接口静态分析与识别模块首先接受固件源码，然后通过 CodeQL 静态分析引擎建立抽象语法树构建的数据库，通过预定义的查询规则从固件源码中提取

驱动相关信息，包括驱动函数接口、驱动 MMIO 操作、依赖数据结构与函数间调用关系等信息，并以统一的数据模型对外提供查询接口。

驱动函数分析、替换与编译模块首先接受驱动函数接口静态分析与识别模块输出的驱动函数信息，然后针对各驱动函数进行类型分类并通过合适的函数替换策略生成可替代实现，之后对生成的替换函数代码做质量检查，如果质量检查通过，则将替代实现集成回工程并驱动构建，进一步通过交叉编译验证替换代码是否存在编译错误。如果存在编译错误，则触发闭环修复智能体进行修复，修复完成后再次通过交叉编译验证替换代码是否存在编译错误。该模块采用 LangGraph 智能体框架实现复杂的推理和决策流程，确保生成的替换代码既满足语义保持性要求，又能够正常通过交叉编译工具链的验证。

动态测试反馈闭环模块首先接受驱动函数分析、替换与编译模块输出的替换代码，然后在仿真环境中运行固件，采集运行日志、崩溃信息等，并将错误信息结构化，触发仿真修复智能体对嵌入式程序进行重新分析与替换代码修正，修正完成后再次通过仿真构建智能体交叉编译生成可执行的二进制固件程序，再次在仿真环境中运行固件，如此循环，直至替换代码通过仿真测试。通过多智能体协作，该模块构建”生成-测试-反馈-修正” 的闭环机制，使固件构建仿真与自修复的全过程实现自动化。

3.1 驱动函数接口静态分析与识别模块设计

本模块是整个方案的基础支撑层，负责自动、精准地定位固件中与硬件交互的代码，并将其抽象为标准化的” 可查询知识库”。模块基于 CodeQL 静态分析引擎实现，通模块的核心职责是建立从源码到硬件依赖信息的映射，识别固件中与硬件交互的函数，确定函数访问硬件的方式（如直接寄存器访问、HAL 库调用或间接句柄操作），并分析函数之间的调用依赖关系。模块采用静态分析技术构建包含语法树、控制流图和数据流图的综合数据库，通过精心设计的查询规则识别 MMIO 结构体实例和字段访问，进而识别涉及硬件访问的驱动函数，为替换模块提供精确的语句定位信息。模块输出的分析结果包含驱动函数列表、MMIO 访问操作、函数调用关系图和数据结构信息，这些结构化的分析结果被存储在统一的数据模型中，为下游模

块提供可查询的知识库。

3.1.1 MMIO 访问模式识别方法

MMIO（Memory-Mapped I/O，内存映射 I/O）是一种将硬件外设的寄存器映射到处理器内存地址空间的机制。通过 MMIO，软件可以通过访问内存地址的方式与硬件外设进行交互，而不需要特殊的 I/O 指令。嵌入式厂商通常使用 C 语言结构体来定义 MMIO 寄存器映射，结构体的每个字段对应一个硬件寄存器，字段的偏移量与硬件规范中的寄存器地址偏移保持一致。

MMIO 结构体的典型代码示例如代码 3.1所示。

```
// STM32 UART寄存器映射示例
typedef struct {
    __IO uint32_t CR1;    // 控制寄存器1
    __IO uint32_t CR2;    // 控制寄存器2
    __IO uint32_t CR3;    // 控制寄存器3
    __IO uint32_t BRR;    // 波特率寄存器
    __IO uint32_t GTPR;   // 保护时间与预分频寄存器
    __IO uint32_t RTOR;   // 接收超时寄存器
    __IO uint32_t RQR;    // 请求寄存器
    __IO uint32_t ISR;    // 中断与状态寄存器
    __IO uint32_t ICR;    // 中断标志清除寄存器
    __IO uint32_t RDR;    // 接收数据寄存器
    __IO uint32_t TDR;    // 发送数据寄存器
} USART_TypeDef;

// 通过强制类型转换创建MMIO实例
#define USART1_BASE    (0x40011000UL)
USART_TypeDef *USART1 = (USART_TypeDef *)USART1_BASE;
```

代码清单 3.1 MMIO 结构体示例

定位 MMIO 结构体实例是识别硬件相关代码的关键步骤。本研究总结出三种典型的 MMIO 实例定位模式，分别如代码 3.2、代码 3.3和代码 3.4所示。

模式一：直接地址强转。这类代码将固定外设基地址强制转换为外设寄存器结构体指针，常见于寄存器定义的头文件中。在代码中呈现为类似 `#define USART1 ((USART_TypeDef *)USART1_BASE)` 的宏定义，后续代码直接对该指针进行字段访问如 `USART1->DR`、`USART1->CR1`。

```
// 驱动源码示例：直接地址强转模式
#define USART1_BASE 0x40011000UL
#define USART1 ((USART_TypeDef *)USART1_BASE)
#define USART2 ((USART_TypeDef *)USART2_BASE)
#define USART3 ((USART_TypeDef *)USART3_BASE)

void uart_init_direct(void) {
    USART1->CR1 = 0x2000;  // 直接对强转实例进行寄存器访问
    USART1->BRR = 0x0341;
```

```
}
```

代码清单 3.2 直接地址强转模式示例

模式二：间接初始化。这类代码通过句柄（handle）、配置表（descriptor/config table）等间接结构，将外设基地址赋值给某个”寄存器块指针成员”，后续通过该成员完成寄存器访问。在 STM32 HAL 生态中，这类成员常命名为 `Instance`。在代码中呈现为初始化时的赋值语句如 `huart->Instance = COM_USART[COM]`，后续寄存器访问呈现为多级解引用如 `huart->Instance->DR`。在基于配置表的驱动中，也可能呈现为 `handle.Instance = spi_config[i].Instance` 这种从配置表复制的形式。

```c
// 驱动源码示例：间接初始化模式（HAL句柄）
UART_HandleTypeDef huart;

void BSP_COM_Init(COM_TypeDef COM) {
    huart.Instance = COM_USART[COM];  // 将外设基地址赋值给Instance成员
    HAL_UART_Init(&huart);
    // 后续访问呈现为多级解引用
    huart.Instance->DR = data;
}

// 驱动源码示例：间接初始化模式（配置表）
typedef struct {
    USART_TypeDef *Instance;
    uint32_t BaudRate;
} SPI_Config;

SPI_Config spi_config[] = {
    {SPI1, 115200},
    {SPI2, 9600}
};

void rt_hw_spi_bus_init(void) {
    for (int i = 0; i < sizeof(spi_config)/sizeof(spi_config[0]); i++) {
        spi_bus_obj[i].handle.Instance = spi_config[i].Instance;
        // 通过配置表中的Instance赋值
    }
}
```

代码清单 3.3 间接初始化模式示例（HAL 句柄和配置表）

模式三：常量指针参数。这类代码通过函数参数接收 MMIO 结构体指针（寄存器块基址），常见于底层驱动或通用外设操作函数。在代码中呈现为函数定义的形参类型是某个寄存器结构体指针，函数体内部通过该参数访问寄存器字段，如 `void mmio_uart_init(USART_TypeDef *uart) { uart->CR1 = 0x2000; }`。调用者可以将模式一或模式二中的 MMIO 实例作为实参传入，如直接调用 `mmio_uart_init(USART1)` 或传入句柄成员 `mmio_uart_init(huart.Instance)`。

```c
// 驱动源码示例：常量指针参数模式
void mmio_uart_init_pointer(USART_TypeDef *uart) {
```

```
    // 通过指针访问MMIO寄存器
    uart->CR1 = 0x2000;
    uart->BRR = 0x0341;
}

// 调用示例
void main(void) {
    // 直接传模式一的强转实例
    mmio_uart_init_pointer(USART1);

    // 传模式二的句柄成员
    mmio_uart_init_pointer(huart.Instance);

    return 0;
}
```

代码清单 3.4 常量指针参数模式示例

三种定位模式的对比如表 3.1所示。

表 3.1 MMIO 结构体实例定位模式对比

模式	描述	代码呈现
直接地址强转	将固定地址强制转换为外设结构体指针	整数常量 → 结构体指针类型转换
间接初始化	通过配置表将外设基地址赋值给句柄	结构体成员赋值为外设宏
常量指针参数	函数以常量指针形式接收外设基址	参数类型为 MMIO 结构体指针

3.1.2 驱动函数定位准则

静态分析模块的核心任务是识别固件中的驱动函数。本研究采用简单直接的定位准则：**凡是包含 MMIO 访问的函数，均定义为驱动函数**。这一定位准则的合理性在于：MMIO 访问是固件与硬件交互的最底层方式，包含寄存器读写的函数必然与硬件强相关，这类函数在仿真环境中无法直接执行，需要进行替换处理；而不包含 MMIO 访问的函数通常是纯软件逻辑，如数据处理、协议解析、状态管理等，在仿真环境中可以直接运行，无需替换。

设 F 为固件中的函数集合，M 为 MMIO 结构体实例集合，$A(f, m)$ 为谓词" 函数 f 访问 MMIO 实例 m 的字段"，则驱动函数集合 D 定义为：

$$D = \{f \in F \mid \exists m \in M : A(f, m)\}$$

驱动函数识别的流程如下：首先通过三种定位模式识别代码中的所有 MMIO 结构体实例；然后对每个 MMIO 结构体实例，追踪其字段访问表达式，记录访问的字段名、访问类型（读/写）、所在语句和所在函数；接着对所有包含 MMIO 字段访问的函数进行汇总，形成驱动函数列表；最后分析驱动函数之间的调用关系，构建函数调用图。

3.1.3 数据流追踪目标

系统以"MMIO 字段访问" 为源点，以" 函数参数、局部变量、返回值" 为汇点，构建数据流配置并追踪路径。追踪目的包括：

1. 确定哪些函数参数最终影响了 MMIO 访问；2. 识别需要保留的业务逻辑与需要替换的硬件操作；3. 为替换模块提供精确的语句定位信息。

3.1.4 统一数据模型设计

系统定义了统一的数据模型来表示分析结果，包括 `FunctionInfo`（函数基本信息）、`MMIOAccess`（MMIO 访问点）等结构，为上层模块提供标准化查询接口。数据模型采用 Python 数据类（dataclass）定义，支持序列化和反序列化。

数据模型包括函数信息模型（包含函数名、文件路径、行号、签名、返回类型、参数列表、分类、源码、调用关系等信息）、MMIO 访问点模型（包含结构体类型、字段名、访问类型、行号、所在函数、上下文代码等）和项目分析结果模型（包含项目路径、CodeQL 数据库路径、所有函数、MMIO 访问点、函数调用图、驱动函数列表、分析时间戳等）。这些数据模型通过 JSON 序列化存储在缓存层，供后续模块查询和使用。

3.2 驱动函数分析、替换与编译模块设计

驱动函数分析、替换与编译模块负责语义理解和代码生成，采用大语言模型驱动的智能体架构，通过 LangGraph 框架构建图式工作流，实现从函数分类、代码生成、质量验证到编译集成的完整自动化流程。该模块采用" 语义分析 → 代码生成 → 质量验证 → 编译集成 → 错误修复" 的迭代优化模式，通过多轮迭代逐步提升替换质量，直至生成能够在仿真环境中正常运行的替换代码。

3.2.1 分析替换与编译智能体工作流设计

工作流以**单个驱动函数**为粒度调度：每轮从静态分析模块或任务队列中取定一条驱动函数记录（含签名、源码片段、MMIO 访问上下文等），在图式状态机中顺序执行，直至该函数被判定为无需替换并结束，或替换实现通过校验与编译并被采纳。为便于叙述与实现解耦，整条流水线在逻辑上划分为**驱动函数分析与替换代码生成**、**替换代码修正与编译集成**两个子模块；前者由分析智能体（Analyzer Agent）主导完成上下文拉取、分类与候选替换生成，后者由构建修复智能体（BuilderFixer）在工程侧完成定义校验、构建与失败修复。

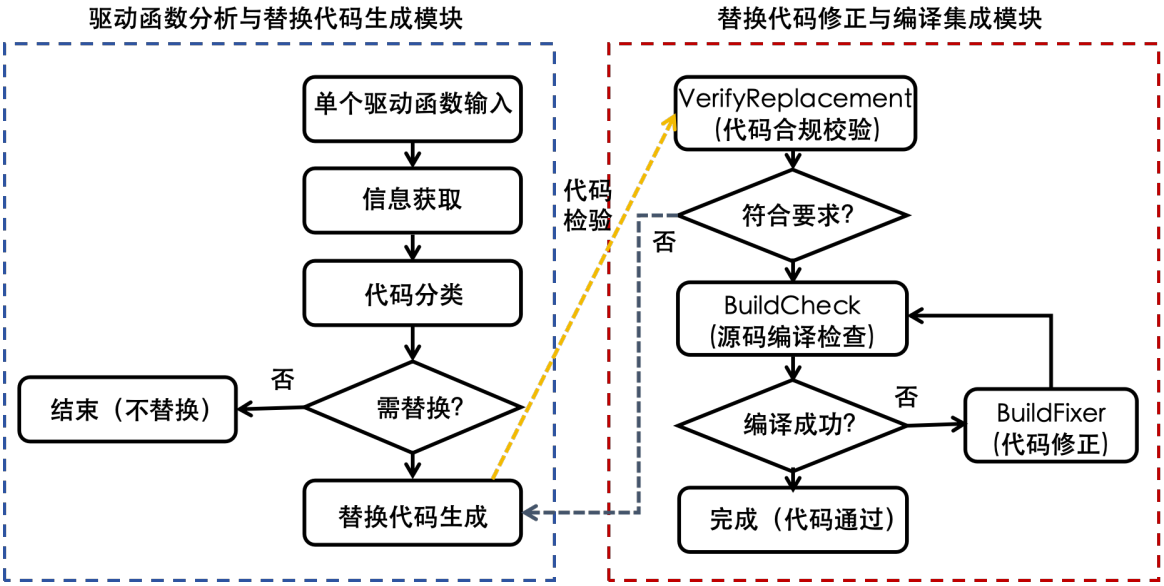

图 3.2 驱动函数分析替换编译模块示意图

3.2.1.1 驱动函数分析与替换代码生成模块设计

该子模块由 **Analyzer Agent** 驱动，在 LangGraph 状态机中通过”工具调用—推理—写回”的循环完成单函数处理。执行顺序固定为：**信息获取** → **代码分类** →（仅在分类结果认定该函数需要替换时）**替换代码生成**。

信息获取阶段通过工具调用拉齐该驱动函数在工程中的可观测上下文，包括完整实现或相关片段、函数签名与参数类型、调用关系与 MMIO 仿真侧提示等（如

GetFunctionInfo、GetFunctionCallStack、GetMMIOFunctionEmulateInfo）。**代码分类**阶段综合源码与静态分析结果，由大语言模型判定函数语义类别及是否属于必须替换的硬件相关行为；分类所依据的类别体系及提示构造方式见第 3.2.2 节。**替换代码生成**仅在" 需要替换" 分支上触发：模型在既定类别上选择替换策略并产出与签名一致的候选实现，具体策略（如 Skip、Strip、Redirect、Event 等）的选择规则与生成约束同样在第 3.2.2 节展开，本节只给出子模块内的先后次序与数据交接关系。

3.2.1.2 替换代码修正与编译集成模块设计

该子模块在工程目录中对上一子模块输出的**候选替换代码**做**二阶段检验与修复**，目标是在真实构建环境下得到可链接、可编译的替换定义。

第一阶段（代码合规校验）调用 VerifyReplacement，依据预置规则与接口契约检查候选替换是否满足替换定义（例如签名一致、禁止引入不兼容的存储类或明显违背仿真假设的写法等），过滤与定义明显不符的实现；未通过时可将反馈交回 Analyzer Agent 侧重新生成或局部修订（图 3.2中以虚线表示返回替换生成环节）。

第二阶段（语法编译检查）在通过定义校验后，将替换写入目标源文件（如 ReplaceFuncs，并保留回滚所需的原定义快照），再调用工程构建脚本执行完整编译链接（如 BuildProject）。若**编译失败**，则进入构建修复智能体（BuilderFixer）路径：由构建侧智能体解析编译器诊断信息，结合 GetFunctionInfo、GetMMIOFunctionInfo 等工具定位首个关键错误，通过 UpdateFunctionReplacement 或 FixFunctionBuildErrors 等动作修订替换代码，并再次触发构建，形成" 编译 → 错误解析 → 修复 → 重新编译" 的闭环，直至编译成功或达到步数上限，从而保证最终被采纳的替换函数能够通过工程编译。

编译错误的典型成因包括函数未声明、类型不匹配、缺少头文件、链接错误，以及替换代码引用了目标芯片 HAL 中不存在的结构体成员或与头文件声明不一致等。例如，STM32F401 的 TIM_HandleTypeDef 无 ErrorCode 成员、RCC_PLLInitTypeDef 无 PLLR 成员时，若替换体仍访问这些成员会导致编译失败；将非静态函数误写为 static 亦会破坏链接一致性。构建修复智能体（BuilderFixer）针对不同诊断采取补声

明、改类型与转换、补 `#include`、删改非法成员访问、核对依赖等策略。提示词上约束智能体在构建失败时优先根据 stderr 锁定**第一个错误**所涉函数再查上下文，避免对多函数无序轮询而浪费步数，从而兼顾修复质量与效率。

3.2.2 驱动函数替换策略设计

为实现语义保持的驱动函数替换，本研究在替换前先对驱动函数进行语义分类，并基于分类结果选择对应替换策略，避免硬件依赖被误删或替换不足导致仿真异常。

3.2.2.1 函数分类策略设计

驱动函数替换的核心挑战在于不同类型的驱动函数与硬件交互的方式和语义要求存在显著差异。采用”一刀切”的统一替换策略无法准确处理各类函数的特殊性，容易导致仿真异常或语义丢失。因此，系统在替换前先对驱动函数进行语义分类，根据函数类别选择最合适的替换策略，从而确保替换后代码的正确性和仿真环境的稳定性。

系统基于大语言模型实现驱动函数的自动分类。LLM 通过读取函数源码、识别 MMIO 访问模式、判断函数主要用途等步骤，将函数分类到预定义的类别。分类策略基于对嵌入式驱动函数硬件交互模式的分析，将函数划分为以下类别，如表 3.2所示。

上述八类函数覆盖了嵌入式固件中与硬件交互的各种典型模式。其中，INIT、RECV、LOOP、IRQ、RETURNOK 和 SKIP 六类函数均直接或间接地涉及 MMIO 寄存器访问，需要通过替换消除硬件依赖；CORE 类函数涉及操作系统核心功能（如系统定时器、线程调度），由仿真器直接提供底层支持，无需替换；NODRIVER 类函数虽然存在于驱动模块中，但其操作对固件状态机无影响或属于纯业务逻辑，同样无需替换。该分类体系的设计兼顾了完备性和实用性，确保每类函数都有明确的替换策略或处理方式。

表 3.2 驱动函数分类体系

缩写	描述	典型函数
INIT	外设初始化，影响固件状态机	HAL_UART_Init, BOARD_BootClockRUN
RECV	从硬件/DMA 读取数据	HAL_UART_Receive, ENET_GetRxFrame
LOOP	轮询等待硬件状态	HAL_UART_WaitOnFlag, FLASH_WaitForLastOperation
IRQ	中断处理/回调函数	HAL_UART_IRQHandler, ENET_TransmitIRQHandler
RETURNOK	纯硬件操作，返回状态值	HAL_GPIO_DeInit, HAL_I2C_IsDeviceReady
SKIP	纯硬件操作，无返回值	HAL_ADC_MspInit, FLASH_FlushCaches
CORE	系统区操作，OS 核心函数	SysTick_Handler, rt_thread_init
NODRIVER	存在的驱动操作对固件状态机无影响	main, GPIO_GetInstance

3.2.2.2 替换策略体系设计

本研究总结出五种通用替换策略，每种策略适用于特定的函数类别和使用场景。表 3.3展示了各类函数与替换策略的对应关系，策略的选择由 LLM 根据函数的语义特征自动决定。

以下逐一阐述各策略的设计原理和适用场景。

Skip 策略：移除硬件交互操作。Skip 策略的核心思想是移除驱动函数中的硬件交互操作，但保留函数签名和返回值，使函数执行路径能够支持固件正常运行。对于初始化类函数，移除硬件寄存器配置逻辑；对于控制类函数，移除控制命令的硬件执行；对于轮询类函数，移除等待硬件状态的循环逻辑，避免仿真环境中的死循环。该策略通过跳过与硬件相关的操作，确保程序在仿真环境中能够顺利执行，同时保留了必要的返回值和函数接口，避免了因硬件依赖导致的阻塞或异常。

Redirect 策略：重定向到仿真 I/O。Redirect 策略使用自定义的注入函数（如 HAL_BE_In、HAL_BE_ENET_ReadFrame、HAL_BE_Out）替代硬件访问，为替换后的驱动代码提供与仿真环境交互的标准接口。对于接收类函数，使用 HAL_BE_In（固定长度）或 HAL_BE_ENET_ReadFrame（变长报文）将仿真输入数据填充到缓冲

表 3.3 替换策略选择指南

函数类别	策略类型	说明
INIT	Skip/Steering	移除硬件寄存器操作，保留结构体初始化，避免仿真环境中的阻塞
RECV	Redirect/Steering	利用 HAL_BE_* 注入函数重定向输入输出，实现数据注入
LOOP	Skip	移除轮询等待逻辑，避免仿真环境中的死循环，直接返回成功状态
IRQ	Event	保留回调函数签名，直接触发事件通知，移除硬件条件判断
RETURNOK	ReturnOK	直接返回成功状态值，保留函数签名，不执行硬件操作
SKIP	Skip	直接跳过函数执行，保留函数签名和调用路径，函数体为空
CORE	NoReplacement	完全不替换，保留原函数代码和调用关系，由仿真器提供底层支持
NODRIVER	NoReplacement	不进行任何替换，保留原函数代码和调用关系

区；对于发送类函数，使用 HAL_BE_Out 重定向输出或在不影响状态时跳过写操作。

Event 策略：事件驱动模拟。Event 策略通过事件队列模拟硬件中断触发。在仿真环境中，硬件中断信号由仿真器生成的事件触发，而非真实的硬件信号。该策略保留中断处理函数中的数据处理和回调触发逻辑，移除硬件中断标志寄存器的操作，通过控制流改写让中断处理函数必然执行到 OS 通知和回调触发分支。

Steering 策略：执行流导向。Steering 策略通过控制流旁路和改写，将因硬件依赖而”走不到正确路径”的执行导向到”仿真可达且语义保持”的分支。该策略适用于复杂场景，例如依赖硬件标志的循环、基于 DMA+RingBuffer 的数据接收状态机等。Steering 策略的核心思想是：当固件因为硬件依赖导致执行路径不可达时，通过控制流改写把执行导向到仿真可用的路径，同时保证数据传输和状态更新的语义一致性。

ReturnOK 策略：直接返回成功。ReturnOK 策略针对 RETURNOK 类函数，这类函数通常是简单的辅助函数，主要功能是设置状态、清理资源或执行简单的检查。

该策略的核心思想是：保留函数签名和必要的参数校验，删除函数体的具体实现，直接返回成功状态（如 HAL_OK）。

3.2.3 替换验证机制设计

替换代码生成后，需要在实际集成到固件工程之前进行严格的质量检查。由于替换代码由大语言模型生成，可能存在语法错误、签名不一致或语义偏移等问题，若未经校验直接写入工程，将导致编译失败或运行时异常。因此，系统设计了多层验证机制，在代码写入工程前尽可能拦截不合格的替换实现，减少后续编译修复的迭代次数，提高整体效率。

系统对生成的替换代码进行以下四层验证：

（1）**语法正确性检查**：通过正则匹配和模式识别验证替换代码的基本语法结构，包括括号匹配、语句完整性、关键字使用等。该检查能够快速过滤掉因 LLM 生成失误导致的大括号不匹配、多余分号等常见语法问题，避免将明显不合格的代码送入后续更耗时的编译环节。

（2）**签名匹配检查**：确保替换函数的签名与原始函数完全一致，包括函数名、返回类型、参数类型和参数名称。签名不一致是最常见的替换错误之一，例如 LLM 可能将 `void *` 参数改写为 `uint8_t *`，或遗漏某个参数。签名检查通过解析原始函数声明和替换代码中的函数定义进行比对，任何不一致都会触发重新生成。

（3）**返回值检查**：确保所有代码路径都正确返回与原函数返回类型匹配的值。对于返回 `HAL_StatusTypeDef` 的函数，每条执行路径都必须返回该枚举类型的合法值（如 `HAL_OK`、`HAL_ERROR` 等）；对于 `void` 函数，确保不包含返回非空值的语句。该检查尤其关注条件分支和循环结构中的返回路径完整性，避免因遗漏返回语句导致未定义行为。

（4）**替换规则检查**：验证替换代码是否遵守替换策略的约束条件，包括函数名未被修改、全局变量和静态变量的声明与原始代码一致、未引入原始工程中不存在的头文件依赖等。此外，该检查还验证替换代码中是否正确使用了仿真注入接口（如 `HAL_BE_*` 系列函数），确保 Redirect 策略的替换代码确实通过这些接口与仿真环境交互。

上述四层验证形成递进式的质量过滤机制：语法检查过滤最基本的格式错误，签名和返回值检查确保函数接口的一致性，替换规则检查验证替换策略的执行规范性。未通过验证的替换代码将被退回 Analyzer Agent 重新生成，并附上具体的错误描述，以便 LLM 在下一轮生成中针对性地修正问题。

3.3 动态测试反馈闭环模块设计

动态测试反馈闭环模块是连接”代码替换”与”仿真验证”的关键环节，负责将经过静态分析和替换处理的固件部署到仿真平台中执行，通过实际运行检验替换代码的正确性，并将运行过程中发现的错误进行迭代修复。该模块的设计遵循”运行 → 诊断 → 修复 → 回归”的闭环思想：首先在基于 Unicorn 引擎的仿真平台上加载并执行替换后的固件，通过多种钩子机制全面采集指令执行、基本块覆盖、函数调用和中断触发等运行时信息；当仿真运行出现崩溃、死循环或内存异常等错误时，仿真执行智能体自动分析运行日志定位问题函数，触发闭环修复智能体生成修复方案，修复后重新构建并再次运行仿真，直至固件能够正常完成执行或达到最大迭代次数。本节依次阐述动态执行环境设计、仿真执行智能体设计、反馈信息结构化设计和迭代优化机制设计。

3.3.1 动态执行环境设计

3.3.1.1 仿真平台架构设计

系统采用基于 Unicorn 引擎的固件仿真框架进行动态测试。Unicorn 是一款轻量级的 CPU 模拟器，支持 ARM、MIPS、X86 等多种架构，本研究使用其 ARM Thumb 模式模拟 ARM Cortex-M 系列微控制器。固件仿真子模块采用双层架构设计，由**仿真引擎层**与 **Hook 机制层**构成。仿真引擎层负责提供指令集执行、内存映射等基础仿真能力；Hook 机制层则在指令流和访存路径上设置回调机制，实现函数入口的拦截和宿主侧逻辑的调用，从而支持驱动替换后注入接口的执行。

仿真引擎层基于 Unicorn ARM 模拟器实现 ARMv7-M Thumb 指令集的模拟执行，包括寄存器管理、内存访问以及固件映像的加载和地址空间映射配置。

Hook 机制层通过代码 Hook、基本块 Hook、内存读/写 Hook 和中断 Hook 等机

制，实现执行过程中的信息采集和干预。该机制主要用于代码覆盖率统计、异常内存访问检测和执行轨迹记录等分析任务。在与替换策略配套时，Hook 机制层承担了关键作用：通过预留 HAL_BE_* 系列注入接口进行函数入口级拦截。具体而言，链接阶段的弱符号桩在宿主机侧被接管，当执行流到达约定入口时，不再在模拟 CPU 上执行原函数体，而是转入宿主机例程，完成缓冲区读写、数据帧搬运或块传输等操作，并按照调用约定返回结果和更新内存状态，随后继续执行。通过这种方式，驱动函数使用 Redirect 策略写入固件的 HAL_BE_* 接口调用可以被固件仿真模块拦截并执行，以支持固件与仿真平台的数据交互，完成固件与仿真平台的协同工作。

表 3.4归纳**固件仿真模块**中与驱动替换配套的注入接口及其使用场景。

表 3.4 固件仿真模块注入接口与使用场景

注入接口	使用场景
HAL_BE_return_0	恒定返回 0，多用作成功码或空操作占位；替换侧用于仅需固定”成功”语义、无需真实硬件操作。
HAL_BE_return_1	恒定返回 1，多用作布尔真或约定非零语义。
HAL_BE_Out	以输出长度表征一次发送语义，常见于类 SEND 的输出重定向策略。
HAL_BE_In	定长接收：将仿真输入写入缓冲区，常见于类 RECV 的输入重定向策略。
HAL_BE_ENET_ReadFrame	在变长上界内填充一帧网络类数据，常见于类 RECV 的输入重定向策略。
HAL_BE_Block_Read	按块大小与块数从虚拟介质读入缓冲区，常见于存储或块设备类替换。
HAL_BE_Block_Write	按块粒度写出缓冲区内容，常见于块设备或大块 I/O 类替换。

3.3.2 仿真执行智能体设计

本节设计仿真运行主控智能体（Emulator Runner Agent），该智能体作为动态测试反馈闭环的核心调度单元，负责在仿真环境中执行固件并采集运行日志、崩溃信息等执行数据。

3.3.2.1 职责定义与工具链设计

Emulator Runner Agent 的核心职责包括以下五个方面：（1）启动仿真运行并实时监控执行状态；（2）分析仿真输出并判断运行是否成功；（3）在运行失败时触发故障诊断，定位问题函数；（4）在修复完成后调用构建智能体（Builder Agent）重新构建；（5）执行回归验证并判断是否需要继续迭代。为实现上述职责，该智能体绑定了三类工具：仿真执行工具（EmulateProject），用于启动固件仿真并采集运行结果；信息查询工具（GetMMIOFunctionEmulateInfo、GetFunctionCallsEmulateInfo），用于获取 MMIO 函数调用日志和函数调用栈信息，支撑故障诊断；以及修复工具和构建工具，通过工具调用接口与闭环修复智能体（Fixer Agent）和闭环构建智能体（Builder Agent）交互。

3.3.2.2 决策逻辑设计

Emulator Runner Agent 的决策逻辑由大语言模型（LLM）驱动，采用条件分支的状态机模型。LLM 根据当前仿真运行的状态和日志信息，在以下决策点进行判断：（1）仿真结果评估——根据退出码、异常循环检测标志等判断运行是否成功，若成功则终止闭环并返回结果；（2）修复需求判断——根据标准输出和标准错误中的错误信息，判断是否需要调用闭环修复智能体（Fixer Agent）进行修复；（3）构建触发判断——在修复完成后，判断是否需要触发闭环构建智能体（Builder Agent）执行重新编译；（4）迭代终止判断——根据当前迭代轮次与最大迭代次数限制的比较结果，决定继续迭代或终止闭环并返回失败报告。该决策机制使得智能体能够根据运行时状态自主调整执行路径，而非依赖预定义的固定流程。

3.3.2.3 仿真执行流程设计

Emulator Runner Agent 的仿真执行流程分为四个阶段。首先为环境准备阶段，Agent 根据目标固件的硬件平台配置 CPU 架构模式和内存映射，包括设置程序计数器（PC）和栈指针（SP）的初始值，以及加载链接脚本和中断向量表等配置信息。其次为固件加载阶段，Agent 将编译生成的 ELF 文件加载到 Unicorn 引擎的虚拟内存

空间中，完成符号解析和地址重定位。第三为执行监控阶段，Agent 启动仿真器运行固件，通过指令级 Hook 和内存访问 Hook 实时采集运行日志、程序计数器变化和内存访问记录，同时监控异常信号（如段错误、非法指令）和运行超时情况。第四为结果分析阶段，Agent 对采集的日志和状态信息进行综合分析，判断运行是否成功，若失败则提取崩溃地址、错误类型和调用栈等关键信息，生成结构化的反馈摘要传递给故障诊断模块。

3.3.3 反馈信息结构化设计

系统在仿真运行过程中采集多种类型的运行信息，用于评估替换效果和指导迭代优化。运行监控采集的信息包括执行状态（成功、失败、超时）、程序计数器的最终值、执行的基本块数量、执行的指令总数、标准输出和标准错误输出等。系统定义了统一的数据结构来表示执行结果，包含上述所有采集的信息，该数据结构作为动态测试反馈模块的输出传递给上游模块。

覆盖率统计用于评估固件代码的执行覆盖程度，系统支持基本块级覆盖率统计，通过块 Hook 记录执行过程中访问的所有基本块的地址。错误分类系统将运行过程中发生的各种错误按照类型进行分类，包括执行崩溃错误（访问无效内存、执行非法指令等）、执行超时错误（死循环、阻塞等待等）以及外设或 MMIO 访问相关的异常。

仿真运行产生的原始日志通常是大量的非结构化文本信息，系统对原始日志进行结构化处理，提取关键信息形成反馈摘要。反馈摘要包含迭代轮次、失败类型、错误地址、崩溃信息、相关驱动函数列表和修复建议等字段。反馈信息结构化处理流程包括日志解析、关键信息提取、相关驱动函数分析和摘要生成四个阶段，通过调用栈分析确定错误发生时正在执行的驱动函数及其调用链。

3.3.4 迭代优化机制设计

当运行失败时，闭环修复智能体（Fixer Agent）被触发进行针对性修复：

1. 分析错误类型（编译失败、链接失败、运行崩溃、运行超时）；2. 定位问题函数；3. 生成修复方案；4. 应用修复并重新构建/运行。

通过”静态分析 → 智能替换 → 编译验证 → 动态测试 → 反馈优化”的完整闭环，系统不断迭代优化替换策略。

3.4 本章小结

本章围绕”基于驱动程序替换的固件仿真”给出了系统级设计方案。系统采用”静态识别 → 智能替换 → 构建验证 → 动态反馈”的分层模块化架构，各模块职责分工和数据流关系如第 3 节所述。在此基础上，本章分别设计了三个核心模块：

（1）静态分析模块设计：设计了基于 CodeQL 的 MMIO 访问模式识别方法，总结出三种典型的定位模式，提出了驱动函数定位准则和数据流追踪方法，构建了统一的数据模型。

（2）替换模块设计：设计了基于 LangGraph 智能体架构的工作流，提出了大语言模型驱动的函数自动分类策略，构建了包含 Skip、Redirect、Event、Steering 等策略的替换体系，设计了多层验证机制和编译集成方案。

（3）动态反馈模块设计：设计了基于 Unicorn 引擎的仿真平台架构，构建了 Emulator Runner Agent，设计了反馈信息结构化机制和迭代优化机制。

该设计实现了”静态识别 → 智能替换 → 构建验证 → 动态反馈”的完整闭环。

4 系统实现

4.1 驱动函数接口静态分析与识别模块实现

4.1.1 CodeQL 查询实现

本节详细阐述分析与识别模块的具体实现技术。模块采用 CodeQL 静态分析引擎作为底层支撑，CodeQL 通过构建包含语法树、控制流图、数据流图的综合数据库，为语义级代码分析提供了强大的查询能力。

4.1.1.1 MMIO 操作定位查询实现

针对三种 MMIO 定位模式，本研究设计了相应的查询规则。查询从 CodeQL 数据库中提取所有符合 MMIO 结构体特征的变量及其初始化表达式。查询的核心设计思想是将”寄存器字段访问”追溯到一个可被 CodeQL 稳定描述的基对象表达式，如代码 4.1所示。

```
import cpp
import libraries.MMIOAnalyzer

from
    Function func,
    InterestingMMIOExpr expr
where
    isInterestingMMIOFunction(func) and
    expr.getEnclosingFunction() = func
select expr.toString(),
       expr.getEnclosingFunction().getName(),
       expr.getFile().getAbsolutePath().toString(),
       expr.getLocation().getStartLine(),
       expr.checkExprType(),
       expr.checkEnclosingType()
```

代码清单 4.1 MMIO 表达式收集查询示例

上述查询通过 `import libraries.MMIOAnalyzer` 引入 MMIO 分析库，从 CodeQL 数据库中提取所有有趣的 MMIO 表达式。查询的输出包括表达式字符串、所属函数名、文件路径、行号、表达式类型和封闭类型等信息。

对于”间接初始化”和”常量指针参数”模式，查询采用数据流追踪策略，通过追踪赋值语句和函数参数来识别 MMIO 实例。常量基址的追踪利用 CodeQL 提供的数据流和污点追踪能力，将”常量大值表达式（外设基址）”追踪到后续的变量访问/解

引用位置，如代码 4.2所示。

```
predicate isMMIOConstantExpr(ConstantExpr src_mmio) {
    exists( DataFlow::Node source, DataFlow::Node sink, Expr varAccess,
            PointerDereferenceExpr deref |
        source.asExpr() = src_mmio and
        isConstantBigValueExpr(src_mmio) and
        sink.asExpr() = varAccess and
        deref.getOperand() = varAccess and
        TaintFlow::flow(source, sink) and
        not sink.asExpr().getFile() instanceof DriverFile
    )
}

predicate isMMIOTracedExpr(Expr e) {
    exists( DataFlow::Node source, DataFlow::Node sink, ConstantExpr src_mmio |
        source.asExpr() = src_mmio and
        isMMIOConstantExpr(src_mmio) and
        sink.asExpr() = e and
        TaintFlow::flow(source, sink) and
        not sink.asExpr().getFile() instanceof DriverFile
    )
}
```

代码清单 4.2 常量基址数据流追踪查询示例

上述查询定义了两个谓词：isMMIOConstantExpr 识别常量大值表达式（外设基址），isMMIOTracedExpr 通过污点追踪将源点追踪到汇点。查询的逻辑可形式化表示为：当存在一个常量大值表达式作为数据流源点，经过数据流传播到达某个变量访问或解引用操作时，该操作即为 MMIO 相关的表达式。

```
// 示例：常量基址在代码中的传播
#define USART1_BASE 0x40011000UL
USART_TypeDef *USART1 = (USART_TypeDef *)USART1_BASE;  // 基址强转
huart->Instance = USART1;                               // 赋值给Instance成员
huart->Instance->DR = data;                             // 最终的字段访问点
```

代码清单 4.3 常量基址在代码中的传播示例

查询通过污点追踪，将上述代码中的 USART1_BASE（源点）追踪到最终的 huart->Instance->DR（汇点），从而识别出整个 MMIO 使用链路，如代码 4.3所示。

对于” 函数第 0 个参数是指针类型，实参是常量大值表达式” 的情况，查询通过函数调用关系和参数类型匹配，定位” 把常量基址传给接受结构体指针的驱动函数” 的调用点，如代码 4.4所示。

```
predicate isMMIOStructure_MK(Struct s){
    exists( FunctionCall call, Expr e, TypedefType typeDef |
        call.getTarget().getParameter(0).getType() instanceof PointerType and
        (
            (call.getTarget().getParameter(0).getType().(PointerType).getBaseType() = typeDef and
             typeDef.getBaseType() = s) or
            call.getTarget().getParameter(0).getType().(PointerType).getBaseType() = s
        ) and
```

```
        call.getArgument(0) = e and
        isConstantBigValueExpr(e)
    )
}
```

代码清单 4.4 函数参数类型匹配查询示例

上述查询通过参数类型匹配（PointerType）和实参的常量大值表达式检查，识别出 MMIO 结构体类型的函数调用点。查询的逻辑可表示为：当函数调用的第 0 个参数类型为结构体指针，且该参数对应的实参是经过数据流追踪的 MMIO 常量表达式时，该调用点即为候选的 MMIO 使用点。

```
// 示例：常量指针参数传递
void uart_send_data(USART_TypeDef *uart, uint8_t *data, uint16_t len);

// 调用时传递MMIO实例
uart_send_data(USART1, buffer, len);     // 直接传强转实例
uart_send_data(huart->Instance, buffer, len);  // 传Instance成员
```

代码清单 4.5 常量指针参数传递示例

查询通过参数类型匹配（USART_TypeDef *）和实参的数据流追踪，识别出上述调用点中的 MMIO 参数传递关系，如代码 4.5所示。

4.1.1.2 驱动函数定位查询实现

驱动函数定位模块的核心任务是根据前述定位准则，从固件源代码中识别出所有驱动函数，并构建驱动函数调用图。模块采用 CodeQL 静态分析技术，通过定义驱动函数的判定规则来实现自动化识别。

模块的核心判定规则是：**包含 DriverFieldAccess 且不包含 MMIOFieldAccess 的函数被定义为驱动函数**。DriverFieldAccess 指的是通过句柄或间接方式访问 MMIO 寄存器块的字段操作，而 MMIOFieldAccess 指的是直接通过 MMIO 结构体指针访问寄存器的字段操作。这一定义确保了识别出的函数是真正需要替换的驱动层函数，而非底层的硬件抽象层函数。判定规则的 CodeQL 实现如代码 4.6所示。

```
class DriverFunction extends Function
{
    DriverFunction() {
        exists(DriverFieldAccess fa | fa.getEnclosingFunction() = this) and
        not exists(MMIOFieldAccess fa | fa.getEnclosingFunction() = this)
    }
}
```

代码清单 4.6 驱动函数判定规则

上述查询定义了 DriverFunction 类，该类继承自 Function，通过检查函数内部是否包含 DriverFieldAccess 且不包含 MMIOFieldAccess 来判定函数是否为驱动函数。

在识别出驱动函数列表后，模块进一步分析驱动函数之间的调用关系，构建函数调用图。调用图以驱动函数为节点，函数调用关系为边，记录调用方、被调用方、调用位置等信息。调用图记录了函数间的调用依赖关系，有助于理解函数的调用上下文和语义关联。

```
from DriverToCall call
select call.toString2(),
       call.getTargetFunction().toString(),
       call.getEnclosingFunction().toString(),
       call.isDirectCall()
```

代码清单 4.7 驱动函数调用关系查询

上述查询从 DriverToCall 关系中提取驱动函数之间的调用关系，输出调用边的字符串表示、目标函数名、封装函数名以及调用是否为直接调用等信息，如代码 4.7所示。

4.1.2 数据模型实现

为便于后续模块使用，静态分析结果被组织为统一的数据模型。数据模型采用 Python 数据类（dataclass）定义，支持序列化和反序列化，如代码 4.8所示。

代码清单 4.8 数据模型定义示例

```
# 示例：数据模型定义
@dataclass
class MmioAccessInfo:
    struct_type: str      # 结构体类型，如"USART_TypeDef"
    field_name: str       # 字段名，如"DR"
    access_type: str      # 访问类型：READ/WRITE
    line_number: int      # 行号
    function_name: str    # 所在函数名
    context_code: str     # 上下文代码

@dataclass
class FunctionInfo:
    name: str             # 函数名
    file_path: str        # 文件路径
    line_number: int      # 行号
    signature: str         # 函数签名
    mmio_accesses: List[MmioAccessInfo]  # MMIO访问列表
```

数据模型包括函数信息模型（包含函数名、文件路径、行号、签名、返回类型、参数列表、分类、源码、调用关系等信息）、MMIO 访问点模型（包含结构体类型、

字段名、访问类型、行号、所在函数、上下文代码等）和项目分析结果模型（包含项目路径、CodeQL 数据库路径、所有函数、MMIO 访问点、函数调用图、驱动函数列表、分析时间戳等）。这些数据模型通过 JSON 序列化存储在缓存层，供后续模块查询和使用。

4.1.3 分析流程实现

静态分析模块的分析流程包括四个主要步骤：CodeQL 数据库构建、查询执行、结果聚合和模型存储。

步骤 1：数据库构建。调用 `codeql database create` 命令构建 CodeQL 数据库，该过程包括源码解析、AST 生成、控制流图和数据流图的构建。数据库构建完成后，可以通过 CodeQL 查询语言对代码进行语义级分析。

步骤 2：查询执行。依次执行前述设计的查询规则，包括 MMIO 表达式收集查询、常量基址数据流追踪查询、驱动函数判定查询和调用关系查询。每个查询返回一组结果，这些结果被转换为 Python 对象供后续处理。

步骤 3：结果聚合。将各个查询的结果进行聚合和关联。例如，将 MMIO 表达式与函数信息关联，构建完整的函数-MMIO 访问映射；将驱动函数与调用关系关联，构建函数调用图。

步骤 4：模型存储。将聚合后的结果存储到统一的数据模型中，并通过 JSON 序列化保存到文件。存储的数据模型包括项目分析结果、函数信息列表、MMIO 访问点列表和调用关系图。这些数据通过文件系统缓存，支持后续模块的快速查询和断点续跑。

4.2 驱动函数分析、替换与编译模块实现

4.2.1 智能体工作流实现

本节阐述基于 LangGraph 框架的智能体工作流实现。LangGraph 是一个用于构建有状态、多智能体应用的框架，它通过图（Graph）来表示应用的状态转换和逻辑流，特别适合本系统这种包含多轮迭代和条件分支的复杂场景。

4.2.1.1 驱动函数分析与替换代码生成模块实现

驱动函数分析与替换代码生成模块的设计逻辑见第 3.2.1.1 节，本节阐述其基于 LangGraph 框架的具体实现。该模块的核心功能是以单个驱动函数为粒度，通过分析智能体（Analyzer Agent）依次完成信息获取、函数分类和替换代码生成三个阶段，实现从原始驱动函数到仿真兼容实现的自动化转换。

分析智能体基于 LangGraph 的 `StateGraph` 类构建，采用图式状态机架构。智能体的核心状态包括当前处理的函数信息、函数分类结果、替换代码、验证结果和错误信息。智能体的转换逻辑由条件函数决定，根据验证结果决定是否需要重新生成替换代码或进入下一个处理阶段。在实现上，Agent 通过 `@tool` 装饰器将 GetFunctionInfo、GetFunctionCallStack、GetMMIOFunctionEmulateInfo 等工具函数注册到 LangGraph 框架中，通过 `compile()` 方法编译为可执行的 `CompiledGraph` 对象，由执行器调度节点的执行顺序并管理状态传递。

4.2.1.2 替换代码修正与编译集成模块实现

替换代码修正与编译集成模块的设计逻辑见第 3.2.1.2 节，本节阐述其具体实现。该模块负责对上一模块输出的候选替换代码进行二阶段检验与修复，由 BuilderFixer 智能体统一完成代码集成、编译构建和错误修复，目标是在真实构建环境下得到可编译、可链接的替换定义。

代码集成机制：系统采用”原位替换”策略将替换代码写入工程。对于每个需要替换的驱动函数，系统在目标源文件中保留原始函数定义的快照（用于回滚），然后将替换实现写入函数体。同时，系统在工程中添加弱符号桩函数文件，为仿真注入接口（`HAL_BE_*` 系列函数）提供链接占位，确保在非仿真环境中不会因未定义符号而导致链接失败。这种设计使得同一个工程既可以在原始硬件环境中编译运行，也可以在仿真环境中使用替换代码运行，无需维护两套代码副本。

编译构建与错误修复流程：BuilderFixer 智能体将替换代码应用到工程后执行编译构建，若编译失败则自动进入错误解析与修复循环。替换代码应用通过 ReplaceFuncs 工具实现，准确定位函数在源文件中的位置、保留注释和格式、生成

回滚快照；工程构建通过 BuildProject 工具执行 CMake 构建，配置 ARM 工具链环境并捕获编译输出。内置的编译错误解析器支持 GCC、Clang、ARMCC 等多种编译器，能够将诊断信息转换为包含错误类型、位置和消息的结构化错误对象。编译失败时，BuilderFixer 智能体自动解析编译器输出的错误信息，定位到具体的替换函数和错误原因，然后生成针对性的修复方案并重新构建，形成”编译 → 错误解析 → 修复 → 重新编译”的闭环。修复策略的选择由 LLM 驱动，LLM 根据错误信息和函数上下文智能地选择补声明、改类型与转换、补 `#include`、删改非法成员访问等策略。BuilderFixer 智能体采用”锁定首个错误”策略，优先修复编译器报告的第一个错误所涉及的函数，避免同时处理多个错误导致修复方案相互冲突。编译错误的典型成因包括：函数未声明或声明不匹配、类型不兼容（如 LLM 将 `uint32_t` 参数改写为 `int`）、缺少头文件包含、替换代码引用了目标芯片外设库中不存在的结构体成员等。每次修复后重新触发编译，直至编译成功或达到最大修复轮次。

4.2.2 替换策略实现

各替换策略的设计意图与语义定义见第 3.2.2 节，本节通过代码示例展示各策略的具体替换效果。

4.2.2.1 Skip 策略实现

Skip 策略适用于两类场景：初始化类函数（INIT）和轮询等待类函数（LOOP）。对于初始化类函数，Skip 策略移除所有硬件寄存器配置操作，仅保留函数签名，使调用路径不被中断，同时通过参数透传确保结构体指针在后续代码中仍然可用。对于轮询等待类函数，Skip 策略移除基于硬件标志位的 while 循环等待逻辑，直接返回成功状态（HAL_OK），避免仿真环境中因硬件标志永远不置位而导致死循环。代码 4.9展示了 Skip 策略在 UART 初始化和轮询函数上的替换效果。

```
// 原始代码（初始化类）
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    huart->Instance->CR1 |= USART_CR1_UE;
}

// 替换代码（Skip 策略）
void HAL_UART_Enable(UART_HandleTypeDef *huart) {
    (void)huart;
}
```

```c

// 原始代码（轮询类）
HAL_StatusTypeDef HAL_UART_WaitOnFlag(UART_HandleTypeDef *huart, uint32_t Flag, uint32_t Timeout) {
    while (!(__HAL_UART_GET_FLAG(huart, Flag))) {
        if (Timeout != HAL_MAX_DELAY) {
            if ((Timeout--) == 0) return HAL_TIMEOUT;
        }
    }
    return HAL_OK;
}

// 替换代码（Skip 策略）
HAL_StatusTypeDef HAL_UART_WaitOnFlag(UART_HandleTypeDef *huart, uint32_t Flag, uint32_t Timeout) {
    (void)huart;
    (void)Flag;
    (void)Timeout;
    return HAL_OK;
}
```

代码清单 4.9 Skip 策略替换示例（初始化类和轮询类）

4.2.2.2 Redirect 策略实现

Redirect 策略适用于接收类函数（RECV），其核心思想是将硬件寄存器轮询替换为仿真注入函数调用，同时保留函数中不影响 I/O 的业务逻辑（如状态字段更新、计数器维护等）。对于定长接收场景，使用 HAL_BE_In 将仿真输入数据填充到目标缓冲区，覆盖原始的寄存器读取逻辑；对于变长报文接收场景（如以太网驱动），使用 HAL_BE_ENET_ReadFrame 在当前有效 buffer 描述符处注入一帧网络数据。代码 4.10展示了 UART 接收函数的替换效果，代码 4.11展示了变长报文接收场景的替换效果。

```c
// 原始代码
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData,
                                   uint16_t Size, uint32_t Timeout) {
    while (!(huart->Instance->SR & USART_SR_RXNE));
    *pData = huart->Instance->DR;
    return HAL_OK;
}

// 替换代码（Redirect 策略）
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData,
                                   uint16_t Size, uint32_t Timeout) {
    // Redirect to simulation input function
    HAL_BE_In(pData, Size);
    // Preserve non-I/O logic
    huart->RxXferCount = 0;
    huart->RxXferSize = Size;
    huart->RxState = HAL_UART_STATE_READY;
    return HAL_OK;
}
```

代码清单 4.10 Redirect 策略替换示例（UART 接收函数）

对于变长报文接收场景（如以太网驱动），代码 4.11展示了在当前有效 buffer 指针处调用 HAL_BE_ENET_ReadFrame 注入数据，完成一次 buffer descriptor 更新后立即返回。

```
// 变长报文接收
curBuffDescrip = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
if (rxBuffer->buffer != NULL && rxBuffer->length > 0) {
    HAL_BE_ENET_ReadFrame(rxBuffer->buffer, rxBuffer->length);
}
rxBdRing->rxGenIdx = ENET_IncreaseIndex(rxBdRing->rxGenIdx, rxBdRing->rxRingLen);
```

代码清单 4.11 Redirect 策略替换示例（变长报文接收）

4.2.2.3 Event 策略实现

Event 策略适用于中断处理和回调函数类（IRQ），其核心思想是绕过硬件中断标志的判断条件，直接触发事件通知和回调函数。在原始代码中，中断处理函数通常通过检查硬件中断标志寄存器（如 ETH_DMA_FLAG_R）来决定是否执行回调，而仿真环境中这些硬件标志永远不会被置位，导致回调逻辑永远无法触发。Event 策略通过移除硬件标志判断，直接调用目标回调函数，使中断处理逻辑能够在仿真环境中正常执行。代码 4.12展示了 ETH 中断处理函数的替换效果。

```
// 原始代码
    if (__HAL_ETH_DMA_GET_FLAG(heth, ETH_DMA_FLAG_R)) {
        HAL_ETH_RxCpltCallback(heth);
    }
    __HAL_ETH_DMA_CLEAR_FLAG(heth, ETH_DMA_FLAG_R);
}

// 替换代码 (Event 策略)
void HAL_ETH_IRQHandler(ETH_HandleTypeDef *heth) {
    // Trigger Rx complete callback event directly
    HAL_ETH_RxCpltCallback(heth);
    // No need to check hardware flags or clear registers
}
```

代码清单 4.12 Event 策略替换示例

4.2.2.4 Steering 策略实现

Steering 策略适用于依赖复杂硬件状态机的场景，其核心思想是通过控制流改写将因硬件依赖而” 走不到正确路径” 的执行导向到” 仿真可达且语义保持” 的分支。典型的应用场景是 DMA+RingBuffer 机制：原始代码通过轮询硬件 buffer descriptor 的状态标志位来遍历接收数据，但在仿真环境中这些标志位不会更新，导致循环无法终止。Steering 策略跳过依赖硬件标志的状态扫描循环，在当前 buffer descriptor 处

直接调用 HAL_BE_ENET_ReadFrame 注入仿真数据，同时保留 Ring 索引更新和状态字段维护逻辑，确保数据传输和状态更新的语义一致性。代码 4.13和代码 4.14展示了 NXP 网卡驱动中 ENET_GetRxFrame 函数的原始代码和替换后的代码对比。

```
// 原始代码：依赖硬件标志扫描 Ring，仿真中会卡死
index = rxBdRing->rxGenIdx;
while (0U == (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_EMPTY_MASK))
{
    if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK)) {
        result = ENET_GetRxFrameErr(curBuffDescrip, &rxFrame->rxFrameError);
        break;
    }
    index = ENET_IncreaseIndex(index, rxBdRing->rxRingLen);
    curBuffDescrip = rxBdRing->rxBdBase + index;
    if (index == rxBdRing->rxGenIdx) break;
}

// 后续：遍历 BD，拷贝数据到 rxFrame
do {
    newBuff = handle->rxBuffAlloc(base, handle->userData, ringId);
    if (newBuff != 0U) {
        rxBuffer = &rxFrame->rxBuffArray[index];
        if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK)) {
            isLastBuff = true;
            rxFrame->totLen = curBuffDescrip->length;
            rxBuffer->length = curBuffDescrip->length - buffLen;
        } else {
            rxBuffer->length = curBuffDescrip->length;
            buffLen += rxBuffer->length;
        }
        // ... buffer 管理、BD 状态更新 ...
    }
} while (!isLastBuff);
```

代码清单 4.13 Steering 策略原始代码示例（NXP ENET_GetRxFrame）

替换后的代码如代码 4.14所示，跳过依赖硬件标志的 while 循环，在当前 buffer descriptor 调用 HAL_BE_ENET_ReadFrame 注入数据，并保留 Ring 更新逻辑。

```
// 替换代码：跳过硬件标志扫描，直接注入数据
curBuffDescrip = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
index = 0;
do {
    newBuff = (uintptr_t)(uint8_t *)handle->rxBuffAlloc(base, handle->userData, ringId);
    if (newBuff != 0U) {
        rxBuffer = &rxFrame->rxBuffArray[index];
        rxBuffer->buffer = (void *)(uint8_t *)address;

        if (0U != (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_RX_LAST_MASK)) {
            isLastBuff       = true;
            rxFrame->totLen  = curBuffDescrip->length;
            rxBuffer->length = curBuffDescrip->length - buffLen;
        } else {
            rxBuffer->length = curBuffDescrip->length;
            buffLen += rxBuffer->length;
        }
    }

    // Steering 核心：注入仿真数据
    if (rxBuffer->buffer != NULL && rxBuffer->length > 0)
        HAL_BE_ENET_ReadFrame(rxBuffer->buffer, rxBuffer->length);
```

```
23
24      // 保留 Ring 更新，防止卡在同一 BD
25      index++;
26      rxBdRing->rxGenIdx = ENET_IncreaseIndex(rxBdRing->rxGenIdx, rxBdRing->rxRingLen);
27      curBuffDescrip     = rxBdRing->rxBdBase + rxBdRing->rxGenIdx;
28  } while (!isLastBuff);
```

代码清单 4.14 Steering 策略替换代码示例（NXP ENET_GetRxFrame）

4.2.2.5 ReturnOK 策略实现

ReturnOK 策略适用于 RETURNOK 类函数，这类函数的主要功能是执行简单的硬件操作并返回状态值（如 HAL_OK 或 HAL_ERROR），如 GPIO 配置、外设复位等。在仿真环境中，这些硬件操作既无实际效果也无必要执行，因此 ReturnOK 策略保留函数签名和必要的参数校验（如空指针检查），直接返回成功状态值，跳过所有硬件相关操作。这样既保证了调用方通过返回值判断执行流程的连贯性，又避免了不必要的硬件依赖。代码 4.15展示了 GPIO 去初始化函数的替换效果。

```
1      if (GPIOx == NULL) return HAL_ERROR;
2      GPIOx->MODER = 0x00000000;
3      GPIOx->OTYPER = 0x00000000;
4      // ... more register operations ...
5      return HAL_OK;
6  }
7
8  // 替换代码 (ReturnOK 策略)
9  HAL_StatusTypeDef HAL_GPIO_DeInit(GPIO_TypeDef *GPIOx) {
10     if (GPIOx == NULL) return HAL_ERROR;
11     (void)GPIOx;
12     return HAL_OK;
13 }
```

代码清单 4.15 ReturnOK 策略替换示例

4.3 动态测试反馈闭环模块实现

4.3.1 仿真平台实现

仿真引擎层基于 Unicorn ARM 模拟器实现 ARMv7-M Thumb 指令集的模拟执行，包括寄存器管理、内存访问以及固件映像的加载和地址空间映射配置。初始化时，系统调用 `uc_open(UC_ARCH_ARM, UC_MODE_THUMB, &uc)` 创建 ARM Thumb 模式的模拟器实例，随后依次完成内存区域映射（Flash、RAM 和外设区域，各区域独立配置读写权限）、ELF 或 BIN 格式固件映像的加载、PC（程序计数器）和 SP（栈指针）等关键寄存器的初始值设置，以及代码钩子、内存钩子和中断钩子的注册，用于监控执行过程和注入回调函数。

在内存映射管理方面，系统定义了 flash、ram、peripheral_ram 等不应合并的受保护内存类型，防止不同区域被错误合并；通过自动识别代码段而非数据段作为 Flash 基址，确保固件正确加载；同时为兼容浮点运算的固件（如图像处理、网络协议栈），启用了 VFP 浮点单元支持。

4.3.1.1 钩子机制

Unicorn 提供了多种类型的钩子（Hook），用于在模拟执行过程中拦截特定事件并执行用户定义的回调函数。本系统主要使用以下四类钩子：

代码钩子（Code Hook）：拦截每条指令的执行，用于指令级跟踪和调试。当 CPU 执行到某条指令时，系统会触发注册的回调函数，传递当前指令地址、指令大小等信息。

基本块钩子（Block Hook）：拦截基本块的执行，用于代码覆盖率统计和控制流分析。基本块是一段连续执行的指令序列，只有一个入口和一个出口。通过基本块钩子，系统可以高效地记录程序执行路径。

内存钩子（Memory Hook）：拦截内存读写操作，用于检测非法内存访问、追踪数据流和监控外设寄存器访问。当程序读写内存时，系统会触发回调函数，传递访问地址、访问类型（读/写）、访问大小和数据内容。

中断钩子（Interrupt Hook）：拦截中断和异常的发生，用于处理外设中断、系统调用和异常情况。当 CPU 触发中断或异常时，系统会调用注册的回调函数，可以自定义中断处理逻辑。

本系统利用这些钩子机制实现了驱动函数的拦截与替换。在仿真初始化阶段，系统通过 `uc_hook_add()` 函数注册上述四类钩子，设置相应的回调函数。代码钩子和基本块钩子用于记录函数调用轨迹和代码覆盖率；内存钩子用于监控外设寄存器访问和检测内存访问异常；中断钩子用于模拟外部中断和异常处理。在与驱动替换策略配合时，钩子机制承担了关键作用：当固件执行到替换后的驱动函数入口时，系统通过钩子拦截执行流，不再在模拟 CPU 上执行原函数体，而是调用宿主机侧的 Python 处理函数，完成缓冲区读写、数据帧搬运或块传输等操作，然后将返回值写

入 Unicorn 的寄存器和内存，并设置 PC 指针为返回地址，继续执行固件。通过这种方式，固件中调用 HAL_BE_* 系列接口时，执行流被拦截并转移到宿主机侧的处理例程，完成仿真数据注入和状态同步后返回固件继续执行，从而实现固件与仿真环境的协同工作。

4.3.1.2 弱符号桩函数

弱符号桩函数是连接固件和仿真环境的桥梁。在链接阶段，这些弱符号桩被替换为仿真环境提供的实现。当固件调用这些函数时，控制权转移到仿真环境，由仿真环境处理函数调用并返回结果。

弱符号桩函数的实现示例：

```
__attribute__((weak)) void HAL_BE_In(uint8_t *buffer, uint16_t size) {
    // 该函数在仿真环境中被重写
    // 实现从仿真输入数据填充缓冲区
}

__attribute__((weak)) void HAL_BE_Out(const uint8_t *data, uint16_t size) {
    // 该函数在仿真环境中被重写
    // 实现将数据输出到仿真环境
}

__attribute__((weak)) void HAL_BE_ENET_ReadFrame(uint8_t *buffer, uint16_t size) {
    // 该函数在仿真环境中被重写
    // 实现从仿真输入数据填充以太网帧
}
```

代码清单 4.16 弱符号桩函数实现示例

在仿真运行时，这些弱符号桩函数通过 Unicorn 的钩子机制被拦截处理。具体实现时，处理器首先通过符号表解析函数的虚拟地址，然后从 Unicorn 的寄存器和内存中读取函数参数，接着调用 Python 实现的 handler 接口完成缓冲区读写、数据帧搬运或块传输等操作，最后将返回值写入 Unicorn 的寄存器和内存并将 PC 指针设置为返回地址，继续执行。通过这种钩子机制，固件中调用 HAL_BE_* 系列接口时，执行流被拦截并转移到宿主机侧的 Python 处理例程，完成仿真数据注入和状态同步后返回固件继续执行，从而实现固件与仿真环境的协同工作。

4.3.1.3 运行日志记录模块

运行日志记录模块负责在仿真执行过程中全面采集和记录程序运行状态信息，为故障诊断和反馈闭环提供数据支撑。该模块通过前述的 Unicorn 钩子机制，将程

序执行过程中的关键事件转化为结构化的日志记录。

在指令级信息采集方面，运行日志记录模块利用代码钩子拦截每条指令的执行，记录当前指令地址、指令大小、寄存器快照等信息。这些指令级信息可用于还原程序的完整执行轨迹，在故障诊断时定位到具体的异常指令。

在基本块级信息采集方面，模块利用基本块钩子记录每个基本块的入口地址和执行次数，据此构建代码覆盖率统计和执行路径分析。通过基本块执行记录，可以判断哪些代码分支被实际执行、哪些路径未被覆盖，从而辅助定位因条件判断依赖硬件状态而未被执行到的分支。

在函数调用信息采集方面，模块通过在驱动函数入口和出口地址处设置代码钩子，记录函数的调用与返回事件，包括调用地址、被调用函数名、参数值和返回值。函数调用序列构成了程序运行的动态调用图，有助于理解函数间的调用上下文关系。

在中断触发信息采集方面，模块利用中断钩子记录仿真过程中触发的所有中断和异常事件，包括中断号、触发地址和处理器状态。中断触发记录对于分析中断驱动型固件的执行流程尤为重要，能够揭示中断处理函数的调用时机和频率。

上述四类日志信息在仿真运行过程中实时采集并统一组织，为后续的故障诊断和反馈闭环提供全面的运行时数据基础。

4.3.2 仿真执行智能体实现

仿真执行智能体（Emulator Runner Agent）基于 LangGraph 框架实现，采用图式工作流架构。该智能体的核心职责包括：启动仿真运行并监控执行状态、分析仿真输出并判断是否需要修复、在运行失败时调用修复智能体进行修复、在修复完成后调用构建智能体重新构建、执行回归验证并判断是否需要继续迭代。仿真执行智能体通过工具调用的方式与其他智能体交互，工具绑定包括仿真工具（EmulateProject、GetMMIOFunctionEmulateInfo、GetFunctionCallsEmulateInfo）、修复工具和构建工具。智能体的决策逻辑由 LLM 驱动，LLM 根据当前的运行状态和日志信息，决定是否需要继续运行、是否需要调用修复工具、是否需要触发构建流程。

4.3.3 反馈闭环实现

4.3.3.1 多智能体架构实现

动态测试反馈闭环由三个智能体协同工作完成。仿真执行智能体作为主控，负责驱动整个闭环流程，协调仿真执行、故障诊断和修复验证。修复智能体负责分析仿真日志、定位问题函数、生成替换代码。构建智能体负责将修复后的代码集成到工程并执行编译。三个智能体通过标准化的数据接口进行交互，形成" 仿真 → 诊断 → 修复 → 构建 → 回归" 的完整闭环。

修复智能体接收仿真执行智能体传递的仿真输出信息，包括标准输出、标准错误、MMIO 函数调用日志和函数调用栈信息。该智能体通过分析这些日志定位出错的替换函数，生成修复方案并调用数据管理接口更新替换代码。修复智能体的输出是结构化的 ReplacementUpdate 对象，包含函数名、替换代码和修复原因。构建智能体接收修复智能体生成的替换代码，调用替换工具将代码应用到源文件，然后执行编译构建。该智能体的输出是 BuildOutput 对象，包含退出码、编译输出和错误信息。

4.3.3.2 闭环协同工作流实现

动态测试反馈闭环的工作流程遵循" 仿真 → 诊断 → 修复 → 构建 → 回归" 的循环模式。工作流程以任务为单位组织，每个任务对应一个固件工程、一个构建配置和一组待替换函数列表。

闭环协同工作流的核心逻辑如算法 4-1所示。该工作流以 LangGraph 框架实现，通过状态管理和条件分支控制执行流程。仿真运行阶段调用 EmulateProject 工具执行固件仿真，采集运行输出、退出码和异常循环检测标志。当仿真成功时，直接返回结果；当仿真失败时，进入故障诊断阶段。

算法 4-1以形式化方式描述了闭环协同工作流的核心逻辑。工作流维护状态 $\sigma = (E, C, \mathcal{F}, \mathcal{P})$，其中 E 为固件工程，C 为构建配置，$\mathcal{F}$ 为待替换函数集，$\mathcal{P}$ 为已处理函数集合。每轮迭代依次执行四个阶段：

阶段 1（仿真运行）：调用 EmulateProject 工具执行固件仿真，采集三类运行日志——标准输出 $\mathcal{L}_{\text{out}}$、MMIO 函数调用日志 $\mathcal{L}_{\text{mmio}}$ 和函数调用栈 $\mathcal{L}_{\text{call}}$。若仿真状态

算法 4-1: 闭环协同工作流核心逻辑

输入: 固件工程 E，构建配置 C，待替换函数集 $\mathcal{F}$，最大迭代次数 K
输出: 仿真结果 $R \in \{\text{Success}, \text{Fail}(\cdot)\}$

```
初始化工作流状态 σ ← (E, C, F, P ← ∅)
// P 为已处理函数集合
for k ← 1 to K do
    // 阶段 1：仿真运行
    (s, L_out, L_mmio, L_call) ← Emulate(σ)
    if s = NORMAL then
        return R ← Success
    else
        // 阶段 2：故障诊断
        f* ← Diagnose(L_out, L_mmio, L_call)
        if f* = ⊥ or f* ∈ P then
            return R ← Fail("不可修复故障")
        else
            P ← P ∪ {f*}
            // 阶段 3：LLM 驱动修复
            Δ_f* ← LLM-Fix(f*, L_out, L_mmio)
            σ ← ApplyPatch(σ, f*, Δ_f*)
            // 阶段 4：增量构建与回归验证
            σ ← Rebuild(σ)
return R ← Fail(max_iter_reached)
```

$s = \text{Normal}$（退出码为零且未检测到异常循环），则返回成功结果。

阶段 2（故障诊断）：基于 $\mathcal{L}_{\text{mmio}}$ 和 $\mathcal{L}_{\text{call}}$ 从崩溃地址或异常循环位置定位故障函数 f^*。若无法定位（$f^* = \perp$）或 f^* 已在 $\mathcal{P}$ 中（表明先前修复未生效），则终止闭环并返回失败。

阶段 3（LLM 驱动修复）：将故障函数 f^* 的源码、上下文和运行日志输入大语言模型，由 LLM 分析故障原因并生成修复补丁 Δ_{f^*}。修复策略包括调整等待条件（添加超时机制）、修正状态管理逻辑（补充遗漏的状态更新）和切换替换策略（如从 Strip 改为 Redirect）。补丁通过 ApplyPatch 写入数据管理器，更新 f^* 的替换实现。

阶段 4（增量构建与回归验证）：构建智能体从数据管理器读取更新后的替换代码，执行增量编译。若编译失败，编译错误反馈至修复智能体进行进一步修复；若编译成功，更新工作流状态 σ，进入下一轮迭代进行回归验证。

系统设置最大迭代次数 $K = 3$，兼顾修复覆盖率与执行效率。通过多智能体协

同和 LLM 驱动的语义修复机制，工作流能够自动处理驱动替换代码与仿真环境之间的适配问题，显著减少人工干预。

4.3.3.3 终止条件与错误处理实现

动态测试反馈闭环的终止条件包括成功终止和失败终止两种情况。成功终止的条件是仿真工具返回 success 为 true，即退出码为零且未检测到异常循环。此时任务进入" 已运行" 状态，表示仿真成功完成。失败终止的情况包括多种：达到最大迭代次数限制，即已进行 3 轮修复但仍然失败；无法从崩溃地址或异常循环位置解析出问题函数；解析出的函数已经在 MMIO 信息列表中，说明该函数已经被处理但仍然失败。

错误处理机制包括编译错误处理和运行时错误处理。编译错误由构建智能体检测，如果编译失败，错误信息通过 BuildOutput 返回给仿真执行智能体，LLM 根据错误信息决定是否需要调用修复智能体进行修复或终止闭环。运行时错误由仿真工具检测，包括内存访问错误、非法指令执行、异常循环等，错误信息包含在标准输出和标准错误中，LLM 分析这些信息定位问题函数。系统还实现了 GraphRecursionError 处理机制，当 LangGraph 框架达到递归深度限制时，系统会分析失败的对话历史，生成详细的故障报告并退出。

通过多智能体协同工作和闭环迭代优化机制，系统能够自动诊断和修复大部分运行时问题，显著减少人工干预的需求。系统记录每次迭代的详细信息，包括失败现象、修复方案、验证结果，便于问题追溯和系统优化。

4.4 本章小结

本章详细阐述了基于驱动程序替换的固件仿真系统的具体实现细节。实现工作围绕第三章提出的设计方案展开，采用 CodeQL、LangGraph、Unicorn 等技术栈，构建了完整的自动化工具链。

在静态分析模块实现方面，本研究基于 CodeQL 引擎实现了 MMIO 访问模式识别、驱动函数定位和数据流追踪等功能。设计了针对三种 MMIO 定位模式的查询规则，构建了统一的数据模型，实现了从源码到硬件依赖信息的自动提取。

在替换模块实现方面，本研究基于 LangGraph 框架实现了多智能体架构。分析智能体负责函数分析和替换代码生成，构建智能体负责代码集成和编译，修复智能体负责错误分析和修复。通过结构化的提示词工程和替换策略实现，系统能够自动生成高质量的替换代码。

在动态反馈模块实现方面，本研究基于 Unicorn 引擎构建了仿真平台。通过弱符号桩函数和 Python 处理器，实现了固件与仿真环境的数据交互。通过多智能体协同工作和闭环迭代优化，系统能够自动诊断和修复运行时问题，显著减少了人工干预。

实现结果表明，系统能够在无人干预的情况下完成从静态分析到动态反馈的完整流程。

5 实验设计与结果分析

本章通过系统性的实验验证本文提出的基于驱动程序替换的固件仿真方法的有效性。实验从多个维度对方法进行评估，包括静态分析模块的识别准确率、运行正确性、方法对比以及应用验证。

5.1 实验设计

为了全面评估本文方法的有效性和实用性，本节介绍实验的整体设计，包括实验环境的搭建、测试用例的选择以及实验目标的定义。

5.1.1 实验环境

实验在以下软硬件环境中进行，具体配置如表 5.1和表 5.2所示。

表 5.1 硬件环境配置

配置项	规格
主机	Intel Xeon Platinum 8350C 处理器（128 核心），503GB 内存，869GB SSD

表 5.2 软件环境配置

配置项	规格
操作系统	Ubuntu 22.04 LTS
编译工具链	ARM GCC 10.3.1、CMake 3.21.4
仿真环境	Unicorn 2.0.2、UnicornAFL（基于 Unicorn 的模糊测试扩展）
模糊测试	AFL++ 4.08c+（基于 v4.08c 的开发版本）
大模型	DeepSeek V3.2

5.1.2 测试用例

为了验证方法的通用性和有效性，本研究构建了涵盖不同芯片平台、操作系统、驱动复杂度和外设类型的固件测试集。需要说明的是，受限于闭源商用固件的可获

取性和法律合规要求，本研究的测试固件均来源于公开渠道，包括芯片厂商官方SDK 示例和开源社区项目。

（1）芯片平台选择

测试固件覆盖了物联网设备中常见的三类主流 MCU 平台：STM32 系列（ARM Cortex-M4/M7）、NXP i.MX RT 系列（ARM Cortex-M7）和 Atmel SAM 系列（ARM Cortex-M4/M7）。

（2）操作系统选择

测试固件覆盖了四种典型的嵌入式软件架构：FreeRTOS、RT-Thread、Zephyr 和裸机。FreeRTOS 是全球使用最广泛的开源实时操作系统，具有内核精简、易于移植的特点，支持任务管理、队列、信号量等功能。RT-Thread 是国产开源实时操作系统，提供完整的设备驱动框架和丰富的软件包生态，在国内物联网领域应用广泛。Zephyr 是 Linux 基金会主导的开源实时操作系统，面向资源受限的物联网设备，提供统一的设备驱动模型和丰富的外设支持，近年来在工业和消费级 IoT 领域快速普及。裸机是指无操作系统支持的固件，所有驱动和应用代码直接运行在硬件上，常见于资源极度受限的物联网设备。

（3）外设类型选择

测试固件涵盖了物联网设备中常见的三类外设驱动，详细信息如表 5.3所示。

表 5.3 外设类型分类

类别	外设类型
通信外设	UART（串口通信）、I2C（总线通信）、SPI（串行外设接口）、ETH（以太网控制器）
存储外设	Flash（闪存控制器）、SDIO（SD 卡接口）
定时器外设	TIM（通用定时器）、RTC（实时时钟）、WDT（看门狗定时器）

（4）测试固件构建

本研究测试固件来源于三类公开渠道：一是从 STM32CubeMX 和 NXP MCUXpresso SDK 提取的官方示例程序，这类程序代表规范的驱动代码编写风格，驱动接口清晰、结构标准化，适合验证方法的基本有效性；二是从 GitHub 和 Gitee 收集的开源

物联网设备项目，如 RT-Thread 的板级支持包（BSP）和社区贡献的应用项目，这类项目代码规模更大、驱动逻辑更复杂；三是 Zephyr 项目的官方示例程序和板级支持包，Zephyr 采用统一的设备驱动模型（DM），其驱动代码风格与 FreeRTOS/RT-Thread 的 HAL 驱动存在显著差异，能够验证方法对不同驱动框架的通用性。需要指出的是，本测试集未包含闭源商用固件，对代码混淆、固件加密等对抗性场景尚未涉及，方法在真实商用固件上的适用性有待进一步验证。

经过筛选和整理，最终构建了包含 35 个固件的测试集，详细信息如表 5.4所示。

表 5.4 测试固件详细信息

平台	OS	测试用例名称	MCU	外设
STM32	RT-Thread	stm32f401/base	F401	多外设
		stm32f401/eth	F401	ETH
		stm32f401/fatfs	F401	SD
		stm32f401/fatfs_sdcard	F401	SD
		stm32f401/i2c	F401	I2C
		stm32f401/uart	F401	UART
STM32	FreeRTOS	HTTP_Server_SockRT	STM32	ETH
		HTTP_Server_Sock_RT	STM32	ETH
		LwIP_StreamingSrv	STM32	ETH
		UART_Hyperterminal	STM32	UART
STM32	Zephyr	eth_dhcpv4_f767zi	F767ZI	ETH
		fs_littlefs_l475	L475	Flash
		i2c_hts221_l475	L475	I2C
		uart_echo_l475	L475	UART
Atmel	Zephyr	eth_dhcpv4_e70	SAM E70	ETH
		fs_littlefs_same54	ATSAME54	Flash
		i2c_shell_same54	ATSAME54	I2C
		uart_echo_same54	ATSAME54	UART
NXP	裸机	FATFS_BareMetal	RT1052	SD
		I2C_BareMetal	RT1052	I2C
		LwIP_BareMetal	RT1052	ETH
		UART_BareMetal	RT1052	UART
NXP	FreeRTOS	FATFS_FreeRTOS	RT1052	SD
		LwIP_FreeRTOS	RT1052	ETH
		UART_FreeRTOS	RT1052	UART
NXP	RT-Thread	imxrt1052/base	RT1052	多外设
		imxrt1052/fatfs	RT1052	SD
		imxrt1052/i2c	RT1052	I2C
		imxrt1052/uart	RT1052	UART
		imxrt1060/base	RT1060	多外设
		imxrt1060/eth	RT1060	ETH
NXP	Zephyr	eth_dhcpv4_rt1060	RT1060	ETH
		fs_littlefs_rt1060	RT1060	Flash
		i2c_fxos8700_lpc55	LPC55S69	I2C
		uart_echo_rt1060	RT1060	UART

共 35 个固件，覆盖 NXP/STM32/Atmel 三种平台，FreeRTOS/RT-Thread/Zephyr/裸机四种操作系统

5.2 驱动函数识别与替换统计

驱动函数识别与分类是整个替换流程的基础环节，分类准确性直接影响后续替换策略的选择和替换代码的质量。本节评估系统对驱动函数的自动识别与替换能力，验证静态分析模块的函数识别准确性和 LLM 的替换代码生成效果。

为了全面评估本文方法的驱动处理能力，本研究统计了每个测试用例的驱动函数识别和替换情况（完整分类统计数据见附录表 A.2）。表 5.5展示了各测试用例的

驱动函数识别与替换统计。

表 5.5 驱动函数识别与替换统计

Demo 名称	识别数	替换数	迭代数	Demo 名称	识别数	替换数	迭代数
NXP_FATFS_BareMetal	67	37	26	stm32f401-st-nucleo/uart	55	39	11
NXP_FATFS_FreeRTOS	65	36	34	LwIP_HTTP_Server_SocketRTOS	281	161	18
NXP_I2C_BareMetal	104	50	45	LwIP_HTTP_Server_Socket_RTOS	304	213	17
NXP_LwIP_BareMetal	105	61	20	LwIP_StreamingServer	281	161	18
NXP_LwIP_FreeRTOS	107	70	9	UART_Hyperterminal_IT	120	87	8
NXP_UART_BareMetal	90	49	33	eth_dhcpv4_sam_e70	188	12	7
NXP_UART_FreeRTOS	91	39	25	fs_littlefs_atsame54	76	19	25
imxrt1052-nxp-evk/base	53	34	18	i2c_shell_atsame54	79	14	11
imxrt1052-nxp-evk/fatfs	53	31	24	uart_echo_atsame54	63	13	13
imxrt1052-nxp-evk/i2c	110	50	33	eth_dhcpv4_mimxrt1060	276	12	5
imxrt1052-nxp-evk/uart	73	41	20	fs_littlefs_mimxrt1060	113	2	2
imxrt1060-nxp-evk/base	65	5	4	i2c_fxos8700_lpc55	150	3	0
imxrt1060-nxp-evk/eth	73	6	5	uart_echo_mimxrt1060	103	2	0
stm32f401-st-nucleo/base	55	32	26	eth_dhcpv4_nucleo_f767zi	260	15	0
stm32f401-st-nucleo/eth	55	35	19	fs_littlefs_disco_l475	50	0	0
stm32f401-st-nucleo/fatfs	56	40	14	i2c_hts221_disco_l475	37	0	0
stm32f401-st-nucleo/fatfs_sdcard	281	161	18	uart_echo_disco_l475	35	0	0
stm32f401-st-nucleo/i2c	55	33	5				
总计	**4029**	**1563**	**513**	**平均**	**115.1**	**44.7**	**14.7**

从统计结果可以看出，本文方法在驱动函数识别和替换方面表现出较高的准确性。35 个测试用例共识别出 4029 个驱动函数，平均每个固件 115.1 个；成功生成 1563 个替换代码，平均每个固件 44.7 个。其中 513 个替换代码经过动态反馈闭环进行了迭代优化，平均每个固件 14.7 个。替换代码生成率为 38.8%（1563/4029），未生成替换代码的函数主要包括两类：一是 NODRIVER 类纯业务逻辑函数（812 个，占 20.2%），这类函数不直接访问硬件寄存器，无需替换；二是 RETURNOK 类简单查询函数（860 个，占 21.3%），这类函数在仿真环境中直接返回成功状态值即可，无需生成替换代码。

5.3 驱动函数分类统计

为了全面评估本文方法的驱动函数分类能力，本研究统计了每个测试用例的驱动函数按类别分布情况（完整统计数据见附录表 A.2）。图 5.1展示了各类驱动函数的占比分布情况。

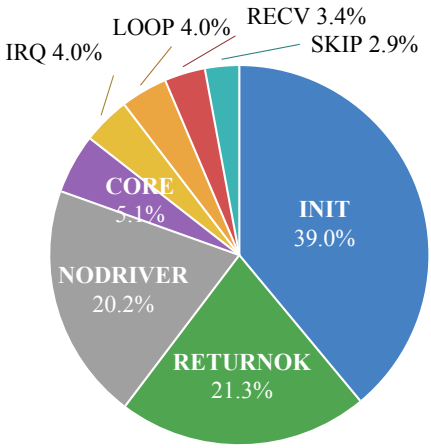

图 5.1 驱动函数分类占比分布

从图 5.1可以看出：

（1）硬件依赖普遍性：4029 个驱动函数中，1563 个（38.8%）生成了替换代码，表明硬件依赖是嵌入式固件普遍存在的特性，验证了驱动函数替换的必要性。NODRIVER 类中 812 个函数（20.2%）被准确识别为纯业务逻辑函数，避免了对非硬件依赖代码进行不必要的替换。RETURNOK 类 860 个函数（21.3%）在仿真环境中直接返回成功状态值，无需生成替换代码，体现了分类策略对简单查询函数的合理处理。

（2）初始化函数占比最高：INIT 类函数占 39.0%（1570 个），其中大部分生成了替换代码，说明固件启动阶段需要大量硬件配置操作，这些函数是替换的重点，必须正确处理以保证固件正常启动。

（3）不同驱动框架的函数分布差异：Zephyr 固件的 NODRIVER 类函数占比显著高于 FreeRTOS/RT-Thread 固件（如 eth_dhcpv4_sam_e70 的 NODRIVER 占 85.1%、eth_dhcpv4_nucleo_f767zi 的 NODRIVER 占 58.8%），这是因为 Zephyr 采用统一的设

备驱动模型（DM），大量设备操作通过抽象层间接访问硬件，直接操作寄存器的驱动函数比例较低。相比之下，基于 HAL 库的 NXP 和 STM32 固件的 INIT 类函数占比更高，反映了 HAL 驱动直接操作外设寄存器的特点。

（4）数据收发函数分布：RECV 类函数占 3.4%（139 个），涵盖 UART、I2C、以太网等多种通信接口。这类函数直接与数据传输相关，替换质量直接影响仿真环境的输入输出能力，是替换工作的关键。

（5）平台差异分析：不同芯片平台的固件在驱动函数构成上存在显著差异。NXP 固件平均每个包含 90.0 个驱动函数，STM32 固件平均 148.8 个，Atmel 固件平均 101.5 个。STM32 固件的驱动函数数量较高，主要原因在于 STM32CubeMX 生成的代码包含更丰富的外设初始化和中间件调用链。

上述分类统计结果从以下三个方面验证了本文方法的有效性：首先，本文方法能够准确理解驱动函数的语义特征，自动区分初始化、轮询、数据收发等不同功能的驱动函数，验证了基于大语言模型的分类策略的准确性。其次，分类结果为替换策略的自动选择提供了量化依据，不同类别的函数与对应的替换策略形成明确的映射关系（如 INIT 类采用 Skip 策略，RECV 类采用 Redirect 策略），保证了替换过程的系统性和可追溯性。最后，分类结果覆盖了 STM32、NXP、Atmel 三种芯片平台以及 FreeRTOS、RT-Thread、Zephyr 和裸机四种操作系统，覆盖 UART、ETH、I2C、Flash、SD 等多种外设类型，驱动函数总数从 53 个到 304 个不等，表明本文方法具有良好的通用性和可扩展性。

5.4 替换功能正确性测试

为了全面评估替换后固件的运行正确性，本研究采用基于关键节点检查的执行轨迹验证方法，以及基于固件执行行为的功能正确性验证方法。

5.4.1 执行轨迹验证方法

执行轨迹验证的目标是确认固件在仿真环境中能够正常执行到反映其核心功能的关键节点。关键节点是指在固件执行过程中能够反映其功能和状态的重要事件，如系统启动、外设初始化、数据收发、任务调度等。通过检查这些关键节点是否被

执行到，以及驱动替换函数的调用行为是否符合预期，可以验证替换代码的功能正确性。

执行轨迹验证方法的步骤是：将测试用例在仿真环境中执行，通过代码钩子记录完整的函数调用轨迹。执行完成后，从调用轨迹中提取关键节点函数的调用事件，验证其是否按预期顺序执行。同时，统计替换驱动函数在仿真运行期间的调用次数，验证替换代码是否被正确触发并正常返回。

本研究从 35 个测试用例中，按外设类型选取代表性样本进行详细的功能正确性验证。选取原则为：每种外设类型（UART、I2C、ETH、FATFS）各选取 2–3 个覆盖不同操作系统（FreeRTOS、RT-Thread、Zephyr、裸机）和不同芯片平台（STM32、NXP、Atmel）的测试用例，共选取 14 个样本。

测试用例的关键节点预期如表 5.6所示。每个测试用例定义了反映其核心功能的关键节点函数，这些函数覆盖了系统启动、外设初始化、数据收发和任务调度等关键阶段。

表 5.6 测试用例与关键节点对照表

Demo 名称	**关键节点**
NXP_FATFS_BareMetal	main, BOARD_InitHardware, BOARD_BootClockRUN
NXP_FATFS_FreeRTOS	osKernelStart, read_task, DbgConsole_Printf
NXP_I2C_BareMetal	main, BOARD_InitHardware, LPI2C_MasterInit
NXP_I2C_FreeRTOS	osKernelStart, i2c_task, LPI2C_MasterTransferNonBlocking
NXP_LwIP_BareMetal	main, BOARD_InitHardware, ethernetif_input, ENET_GetRxFrame
NXP_LwIP_FreeRTOS	osKernelStart, eth_rx_thread, ENET_GetRxFrame
NXP_UART_BareMetal	main, BOARD_InitHardware, LPUART_Init
NXP_UART_FreeRTOS	osKernelStart, uart_task, LPUART_TransferSendNonBlocking
imxrt1052-nxp-evk	osKernelStart, main, rt_pin_write, GPIO_PinWrite
imxrt1060-nxp-evk	osKernelStart, main, rt_pin_write, GPIO_PinWrite
stm32f401-st-nucleo/eth	rtthread_startup, eth_rx_thread_entry, HAL_ETH_ReadData
stm32f401-st-nucleo/fatfs	rtthread_startup, fatfs_thread_entry, rt_pin_write
UART_Hyperterminal_IT	main, HAL_UART_RxCpltCallback, HAL_UART_Transmit_IT
LwIP_HTTP_Server_Socket_RTOS	osKernelStart, ethernetif_input, HAL_ETH_ReadData
总计：14 个测试用例，43 个关键节点，匹配率 100.0%	

实验结果表明，14 个代表性测试用例的关键节点匹配率和替换驱动函数行为验证结果均为正常。总体关键节点匹配率为 100.0%（44/44），表明替换后的固件能够正确执行主要功能路径。

5.4.2 行为验证

关键节点匹配验证了固件主流程的可达性，但替换代码本身是否正确执行仍需进一步验证。本研究从三个层面对替换驱动函数进行行为验证：第一，检查替换函数是否被正确调用，即仿真运行期间替换函数的调用次数是否与预期一致；第二，检查替换函数是否正常返回，未导致仿真崩溃、死循环或异常中断；第三，检查替换函数的返回值和副作用是否与原函数的仿真语义一致（如 Init 策略函数是否正确初始化了仿真数据结构，Redirect 策略函数是否正确完成了数据注入）。

根据表 5.5的统计，35 个测试用例共识别出 4029 个驱动函数，其中 1563 个生成了替换代码。基于上述验证方法，14 个代表性测试用例的行为验证结果如表 5.7所示。

表 5.7 替换驱动函数行为验证结果

测试样本	功能类型	执行到的替换函数数	验证结果
NXP_FATFS_BareMetal	文件系统（FatFS）	10	正常
NXP_FATFS_FreeRTOS		10	正常
NXP_I2C_BareMetal	通信外设（I2C）	4	正常
NXP_I2C_FreeRTOS		4	正常
NXP_LwIP_BareMetal	网络功能（ETH）	13	正常
NXP_LwIP_FreeRTOS		15	正常
NXP_UART_BareMetal	串口功能（UART）	1	正常
NXP_UART_FreeRTOS		6	正常
imxrt1052-nxp-evk	基础功能（GPIO）	9	正常
imxrt1060-nxp-evk		4	正常
stm32f401-st-nucleo/eth	网络功能（ETH）	56	正常
stm32f401-st-nucleo/fatfs	文件系统（FatFS）	17	正常
UART_Hyperterminal_IT	串口功能（UART）	38	正常
LwIP_HTTP_Server_Socket_RTOS	网络功能（ETH）	53	正常

验证结果表明，14 个测试用例中被调用到的替换驱动函数均正常返回，未导致仿真崩溃、死循环或异常中断。未被调用到的替换驱动函数主要属于备用初始化路径、异常处理分支等非主流程代码，不影响固件的核心功能运行。以下以 NXP_LwIP_BareMetal 为例，通过仿真执行调用轨迹展示替换代码在系统启动、协议栈初始化和数据帧接收等阶段的具体行为。

5.4.3 案例分析：NXP_LwIP_BareMetal 以太网数据接收

NXP_LwIP_BareMetal 是基于 LwIP 协议栈的裸机 HTTP 服务器测试用例，运行在 NXP i.MX RT1052 平台上，共识别出 105 个驱动函数，其中 61 个生成了替换代码。该用例覆盖了系统启动、以太网控制器初始化、LwIP 协议栈配置和数据帧接收等完整执行流程，具有较好的代表性。

仿真执行从系统复位开始，依次经过以下关键阶段（摘自函数调用轨迹日志）：

阶段 A：系统启动与协议栈初始化（指令周期 1 – 136623），调用链如清单 5.1所

示。

```
Reset_Handler -> SystemInit -> _start -> main
  -> BOARD_InitHardware -> time_init -> SysTick_Config
  -> SILICONID_ConvertToMacAddr -> SILICONID_ReadUniqueID
  -> CLOCK_GetFreq -> CLOCK_InitExternalClk
  -> lwip_init -> netif_add -> netif_set_addr
  -> netif_set_default -> ethernetif_init -> ethernetif_plat_init
    -> ENET_GetDefaultConfig -> ENET_Init -> ENET_Up
      -> ENET_SetHandler -> ENET_SetTxBufferDescriptors
      -> ENET_SetRxBufferDescriptors -> ENET_SetMacController
    -> ethernetif_phy_init -> PHY_GetLinkStatus
    -> probe_link_cyclic -> ethernetif_probe_link
  -> netif_set_up -> netif_issue_reports -> etharp_request
  -> httpd_init -> httpd_init_pcb
    -> tcp_new_ip_type -> tcp_bind -> tcp_listen_with_backlog
    -> tcp_accept
```

代码清单 5.1 NXP_LwIP_BareMetal 系统启动与协议栈初始化调用链

上述调用链中，BOARD_InitHardware、SysTick_Config、CLOCK_InitExternalClk 等硬件初始化函数由 Skip 策略替换，直接返回成功状态；ENET_Init、ENET_Up、ENET_SetHandler 等以太网控制器驱动函数由 Init 策略替换，完成仿真环境中的数据结构初始化（如缓冲区描述符、MAC 地址配置）；SILICONID_ReadUniqueID 由 ReturnOK 策略处理，返回模拟的芯片唯一标识。整个初始化过程共执行约 136,000 条指令，无崩溃或异常中断。

阶段 B：数据接收与协议栈处理（指令周期 137237 – 137392），调用链如清单 5.2所示。

```
main -> ethernetif_input
  -> fetch_received_pkts
    -> ethernetif_linkinput
      -> ENET_GetRxFrame
        -> ethernetif_rx_alloc -> sys_arch_protect
          -> DisableGlobalIRQ -> EnableGlobalIRQ
        -> HAL_BE_ENET_ReadFrame
        -> ENET_IncreaseIndex
      -> ethernetif_rx_frame_to_pbufs
        -> pbuf_alloced_custom -> pbuf_init_alloced_pbuf
    -> ethernet_input
      -> ethernetif_rx_release -> ethernetif_rx_free
    -> ethernetif_linkinput
      -> ENET_GetRxFrame
        -> HAL_BE_ENET_ReadFrame
```

代码清单 5.2 NXP_LwIP_BareMetal 以太网帧接收与协议栈处理调用链

该阶段是验证替换代码功能正确性的核心环节。调用链显示：(1)ethernetif_input 作为 LwIP 的轮询入口函数被正常触发，表明 Loop 策略的替换代码正确处理了

轮询等待逻辑（通过超时机制避免死循环）；（2）ENET_GetRxFrame 内部调用 HAL_BE_ENET_ReadFrame，该函数是本文方法通过 Redirect 策略生成的仿真注入接口，将硬件帧接收操作重定向为从仿真缓冲区读取数据，日志表明成功接收了两帧以太网数据；（3）接收到的数据帧通过 ethernetif_rx_frame_to_pbufs 正确转换为 LwIP 的 pbuf 结构，并传递至 ethernet_input 进行协议栈解析，最终由 ethernetif_rx_release 释放缓冲区，内存管理逻辑完整闭环。

上述分析表明，本文方法生成的替换代码能够正确支撑从系统启动、协议栈初始化到数据帧接收的完整执行流程，其中 Init、Skip、Loop、Redirect 四类替换策略各司其职，协同实现了对以太网控制器硬件依赖的完整解耦。

5.5 反馈闭环有效性测试

为验证动态测试反馈闭环模块的有效性，本节通过对比分析评估该模块对仿真成功率的影响。实验思路为：统计各测试用例在反馈闭环阶段触发替换更新的函数数量，分析这些更新对仿真执行的实际贡献。

5.5.1 替换更新统计

为评估反馈闭环对不同厂商和不同外设类型的影响，图 5.2以四宫格形式展示了四个主要外设类型（UART、I2C、ETH、Flash）按厂商分组的反馈闭环统计数据（各厂商取测试用例均值）。每个子图中，蓝色柱子表示函数分析总数（左轴），红色柱子表示替换更新总数（右轴），绿色柱子表示新增覆盖函数（右轴）。完整统计数据见附录表 A.1。

统计结果表明：35 个测试用例中，28 个（80.0%）触发了至少一次替换更新，累计 513 次。其中 154 个函数属于” 闭环新增覆盖”——这些函数在静态分析阶段未被识别为需要替换（分类为 NA 或 NODRIVER），但在仿真运行过程中暴露出硬件依赖问题，由反馈闭环驱动生成替换代码。若去除动态反馈闭环模块，这 154 个函数将缺失替换，导致对应的仿真执行路径受阻。

从平台维度分析，Atmel（Zephyr）平台的闭环新增覆盖函数占比最高（39/62，62.9%），主要原因在于 Zephyr 采用统一的设备驱动模型（DM），大量硬件操作通过

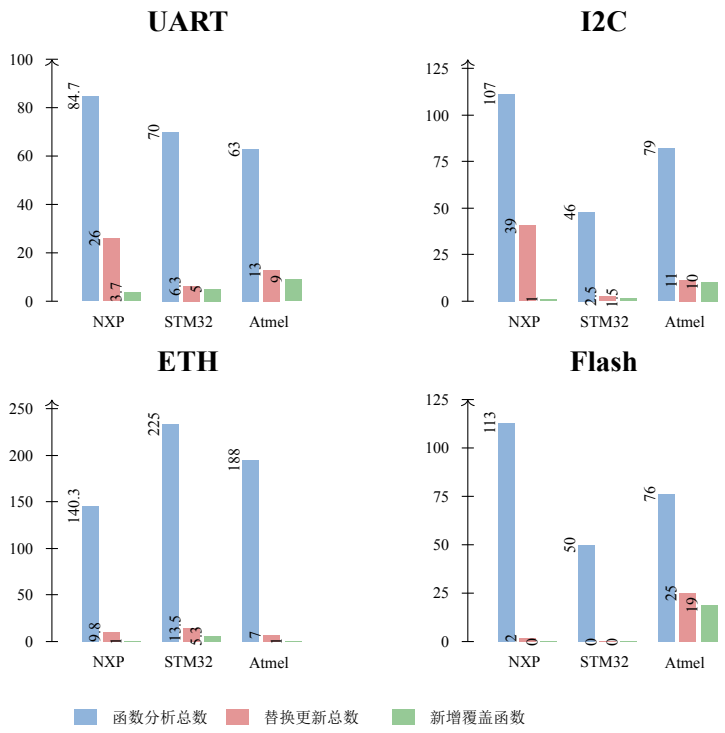

图 5.2 按外设类型分组的反馈闭环统计（各厂商均值）

抽象层间接进行，静态分析阶段难以准确识别底层硬件依赖；STM32（RT-Thread）平台次之（52/93，55.9%），其 BSP 代码中包含较多的平台抽象层函数和中断处理路径，需要运行时反馈才能精确定位。从外设维度分析，I2C 和 UART 类固件的替换更新均值较高，说明串行通信外设的静态分析误分类问题更为突出；ETH 类固件的函数分析总数均值最大，但替换更新比例相对较低，表明网络协议栈的驱动函数识别准确率较高。

5.5.2 典型案例分析

本节通过具体案例分析，深入验证动态反馈闭环在静态分析误分类、硬件依赖识别和平台适应性等方面的实际效果。

案例 1：静态分析误分类的运行时修正。在 NXP_I2C_BareMetal 的仿真执行过程中，反馈闭环检测到固件在 `CLOCK_SetDiv` 函数处触发异常循环。该函数定义于 NXP MCUXpresso SDK 的 `fsl_clock.h`（第 1429 行），负责配置时钟分频器，其核心逻辑如代码清单 5.3所示。

```
1  static inline void CLOCK_SetDiv(clock_div_t divider, uint32_t value)
2  {
3      uint32_t busyShift;
4      busyShift                    = CCM_TUPLE_BUSY_SHIFT(divider);
5      CCM_TUPLE_REG(CCM, divider) = (CCM_TUPLE_REG(CCM, divider)
6          & (~CCM_TUPLE_MASK(divider)))
7          | (((uint32_t)((value) << CCM_TUPLE_SHIFT(divider)))
8          & CCM_TUPLE_MASK(divider));
9      assert(busyShift <= CCM_NO_BUSY_WAIT);
10     if (CCM_NO_BUSY_WAIT != busyShift)
11     {
12         // BUG: wait for hardware handshake - infinite loop in emulation
13         while ((CCM->CDHIPR & ((uint32_t)(1UL << busyShift))) != 0UL)
14         {
15         }
16     }
17 }
```

代码清单 5.3 CLOCK_SetDiv 原函数（NXP_I2C_BareMetal，第 1429 行）

该函数在静态分析阶段被 LLM 分类为 NODRIVER（纯业务逻辑函数），未生成替换代码。然而其实际行为包含典型的 LOOP 类特征：在时钟分频寄存器写入后，通过 `while` 循环轮询 `CCM->CDHIPR` 硬件握手完成标志位。在仿真环境中，该标志位永远不会被硬件置位，导致仿真卡死。

反馈闭环的诊断流程如下：仿真器检测到异常循环后，从崩溃地址解析出故障函数 `CLOCK_SetDiv`，LLM 分析其源码上下文后识别出硬件轮询等待模式，生成替换代码将 `while` 循环体注释掉，同时保留寄存器写入操作以确保时钟配置语义完整。类似的修正同时应用于 `CLOCK_SetMux` 函数（同为 NXP 时钟配置函数，包含相同的硬件握手等待模式）。该案例表明，静态分析阶段的 LLM 分类存在一定误判率，反馈闭环作为运行时保障机制能够有效弥补这一不足，提升系统的整体鲁棒性。

案例 2:反馈闭环策略边界与 CORE 函数人工修复。在 stm32/LwIP_StreamingServer（FreeRTOS）的仿真执行过程中，调度器启动阶段触发异常。具体而言，`vTaskStartScheduler` 调用 `xPortStartScheduler` 后，该函数首先执行一组 Cortex-M 中断优先级一致性检查，如代码清单 5.4所示。

```
1  BaseType_t xPortStartScheduler( void )
2  {
3      // ... vector table checks omitted ...
4      #if ( configASSERT_DEFINED == 1 )
5      {
6          *pucFirstUserPriorityRegister = portMAX_8_BIT_VALUE;
7          ucMaxPriorityValue = *pucFirstUserPriorityRegister;
8          ucMaxSysCallPriority = configMAX_SYSCALL_...
9                               & ucMaxPriorityValue;
10         configASSERT( ucMaxSysCallPriority ); // STUCK here
11         configASSERT( ( configMAX_SYSCALL_...
```

```
                        & ~ucMaxPriorityValue ) == 0U );
    }
    #endif
    // ... if assert passes, start first task ...
}
```

代码清单 5.4 xPortStartScheduler 中的中断优先级一致性检查（FreeRTOS port.c）

在仿真环境中，NVIC 中断优先级寄存器的读回值与真实硬件不一致，导致 `ucMaxSysCallPriority` 为零，触发 `configASSERT` 失败（代码清单 5.4中标注 STUCK here 的行），调度器无法进入首任务路径，仿真卡死。

反馈闭环正确检测到该故障并定位至 `xPortStartScheduler` 函数，但由于该函数被静态分析模块分类为 CORE 类型（FreeRTOS 调度器核心函数），按替换策略设计，CORE 类函数禁止自动替换。因此系统未自动生成替换代码，而是输出错误报告后程序异常退出，等待人工介入。

本文选择手动修复而非放开 CORE 自动替换，基于以下工程考量：CORE 函数位于操作系统内核调度关键路径，涉及上下文切换、中断优先级管理、栈帧初始化等与硬件深度耦合的底层机制，贸然由 LLM 修改可能破坏内核状态一致性，引入难以排查的潜在风险。手动修复策略为：保留 `xPortStartScheduler` 函数本体不变，人工编写一份绕过 NVIC 优先级探测逻辑的 stub 代码，将其硬编码注入所有基于 FreeRTOS 的测试用例，后续这些 demo 不再经过反馈闭环处理该函数。这一工程妥协虽然牺牲了对 CORE 函数的自动化覆盖，但有效维护了内核语义完整性，避免了”为跑通仿真而破坏内核”的过度自动化风险。

该案例作为反馈闭环的特殊失败场景，揭示了自动化替换的合理边界：反馈闭环并非对所有故障都能自动修复，对于涉及操作系统内核关键路径的高风险函数，人工决策仍不可替代。

5.5.3 有效性分析

综合上述统计和案例分析，动态反馈闭环模块的必要性体现在以下三个方面：

（1）覆盖增量：154 个函数（占总替换覆盖的 9.9%）仅在反馈闭环阶段才被识别并生成替换代码。这些函数主要包括两类：一是静态分析分类为 NODRIVER 但实际包含硬件依赖的抽象层函数（如 Zephyr 平台的 `soc_port_configure`）；二是静态

分析未覆盖但仿真执行中暴露问题的辅助函数。

（2）质量修正：359 次替换更新（占总更新次数的 70.0%）属于对已有替换代码的质量修正，包括修复编译错误（签名不匹配、宏展开问题）、调整替换策略（从过度简化恢复为保留关键语义）和补充遗漏的状态管理逻辑。这些修正不增加覆盖数量，但直接决定仿真能否成功执行。

（3）平台差异适应性：不同平台的固件对反馈闭环的依赖程度不同。Atmel（Zephyr）和 STM32（RT-Thread）平台的闭环新增覆盖比例较高（62.9% 和 55.9%），NXP（HAL）平台相对较低（17.8%），这与各平台驱动代码的抽象层次和编码风格一致。

5.6 方法效率评估

本节从人工成本和计算资源消耗两个维度评估本文方法的效率，并与传统方法进行对比分析。

5.6.1 对比方法

本研究选择以下两种具有代表性的方法作为对比，它们与本文方法在核心思想上相似，但所需的人力成本和工作量存在显著差异：

（1）Halucinator：基于驱动替换的固件仿真框架，通过替换目标固件中的 HAL 函数调用，实现对硬件依赖的解耦。用户需要手工编写每个 HAL 函数的替代实现，这些实现通常返回模拟数据或执行简单的状态转换。Halucinator 的优势在于概念清晰、可控性强，但其局限性在于需要大量的人工分析和编码工作，每个新固件都需要重新编写对应的替换层代码。

（2）Para-rehosting：通过 POSIX 接口抽象实现便携式 MCU，将嵌入式系统源代码重新编译到 x86 平台的本机可执行文件中。该方法允许利用 x86 平台上丰富的测试工具进行固件分析。与本文方法相比，Para-rehosting 的核心差异在于需要获取固件样本的源代码，这在第三方的固件分析工作中往往是困难甚至完全不可行的。此外，Para-rehosting 需要实现完整的便携式 MCU 抽象层，工作量大且难以覆盖所有外设功能。

5.6.2 测试方法

从测试集中选取 8 个具有代表性的固件，覆盖 NXP、STM32、Atmel 三种平台，裸机、FreeRTOS、RT-Thread、Zephyr 四种操作系统，以及 UART、I2C、ETH、FATFS 四种外设类型，如表 5.8所示。

对于本文方法，基于 LLM 调用日志统计各固件的 Token 消耗和处理时长。实验使用 DeepSeek V3.2 模型，基准测试表明平均每次 LLM 调用消耗约 12742 Token（含 Prompt 12517 Token 和 Completion 225 Token），平均耗时约 10.7 秒，平均每个驱动函数需要约 11.2 次调用（含初始分析和反馈修正）。对于 Halucinator 和 Para-rehosting，由熟悉嵌入式开发的工程师执行驱动处理工作，统计实际处理的驱动函数数量和人工投入时间。

5.6.3 测试结果

方法效率评估结果如表 5.8所示。左表展示了本文方法的计算资源消耗指标（驱动函数数量、Token 用量、平均处理时长、LLM 总处理时长、人工介入时间），右表展示了 Halucinator 和 Para-rehosting 的人工成本指标（处理的驱动函数数量和人工投入时间）。

表 5.8 方法效率评估

	本文方法					Halucinator		Para-rehosting	
测试用例	驱动数	Token(万)	均长 (s/函数)	LLM(h)	介入 (h)	驱动数	人工 (h)	驱动数	人工 (h)
NXP_FATFS_BareMetal	67	958	120	2.2	0.5	12	2.5	14	4.0
NXP_I2C_BareMetal	104	849	84	2.4	0.5	18	3.5	20	5.5
NXP_LwIP_BareMetal	105	1502	120	3.5	1.0	20	4.0	22	6.5
stm32f401-st-nucleo/fatfs	56	801	120	1.9	0.5	8	2.0	10	3.5
stm32f401-st-nucleo/i2c	55	787	120	1.8	0.5	7	1.5	9	2.5
UART_Hyperterminal_IT	120	1716	120	4.0	0.5	15	3.0	16	5.0
atmel/eth_dhcpv4_e70	188	2689	120	6.3	1.0	25	5.0	28	8.0
atmel/fs_littlefs_same54	76	1087	120	2.5	0.5	12	2.5	14	4.0
平均	**96.4**	**1299**	**116**	**3.1**	**0.6**	**14.6**	**3.0**	**16.6**	**4.9**
总计	**771**	**10389**		**24.6**	**5.0**	**117**	**24.0**	**133**	**39.0**

从表 5.8可以看出：

（1）驱动处理数量：本文方法平均处理 96.4 个驱动函数，约为 Halucinator（14.6 个）的 6.6 倍和 Para-rehosting（16.6 个）的 5.8 倍。这一差异源于分析方式的不同：本文方法在静态分析阶段即对整个驱动库进行全面的函数级扫描，通过 CodeQL 识别所有直接访问 MMIO 地址的函数，在仿真运行前一次性完成所有驱动函数的替换，不存在遗漏。而 Halucinator 和 Para-rehosting 采用的是运行时逐步调试的方式，在仿真执行过程中遇到崩溃或异常时才定位并修复相应的驱动函数，因此表中的驱动处理数量仅反映了调试过程中实际暴露并处理的部分，而非完整的驱动依赖集合。本文方法通过前置的全面静态分析，有效避免了这一问题。

（2）人工投入时间：本文方法 8 个固件的人工介入总时长仅为 5.0 小时（平均 0.6 小时/固件），这里的" 介入" 指工具自动处理失败时需要人工排查和修正的时间，不包括 LLM 自动处理的 24.6 小时。相比之下，Halucinator 需要 24.0 小时（平均 3.0 小时/固件），Para-rehosting 需要 39.0 小时（平均 4.9 小时/固件），且这些时间均为纯人工工作量。本文方法的人工介入仅为 Halucinator 的 20.8% 和 Para-rehosting 的 12.8%。本文方法的人工介入主要集中在固件仿真过程中反馈闭环失败后的问题排查，包括定位失败原因、分析 LLM 生成的替换代码质量、手动修正替换策略等。；驱动函数分析、替换代码生成以及反馈闭环迭代优化均由 LLM 自动完成，无需人工介入。

（3）计算资源消耗：本文方法 8 个固件的 LLM 总处理时长为 24.6 小时（平均 3.1 小时/固件），总 Token 消耗约为 1.04 亿。平均每个驱动函数需要约 116 秒的 LLM 处理时间和约 135 万 Token。计算资源消耗与固件的驱动函数数量正相关：驱动函数最多的 atmel/eth_dhcpv4_e70（188 个）消耗 2689 万 Token 和 6.3 小时，而最少的 stm32f401-st-nucleo/i2c（55 个）仅消耗 787 万 Token 和 1.8 小时。需要指出的是，LLM 处理时间均为自动化执行时间，不占用研究人员的工作时间，与人工成本不具有直接可比性。

（4）综合效率对比：从总时间投入角度分析，本文方法的人工介入时长（5.0 小时）显著低于传统方法（Halucinator 24.0 小时、Para-rehosting 39.0 小时），而 LLM 自动化处理时长（24.6 小时）与 Halucinator 的人工时长相当。考虑到 LLM 处理可以完全自动化执行、支持并行批处理且不占用研究人员时间，本文方法在总人力投

入上具有显著优势。此外，本文方法处理的驱动函数数量（771 个）远超传统方法（Halucinator 117 个、Para-rehosting 133 个），实现了更全面的驱动覆盖。

5.7 模糊测试

本节评估本文方法构建的仿真环境在实际安全测试场景中的性能表现。通过在仿真环境中运行 AFL++ 模糊测试器，评估固件的执行速度、代码覆盖率以及漏洞发现能力。

5.7.1 测试方法

从测试集中选取 9 个具有代表性的固件，覆盖 ETH（3 个）、UART（3 个）、I2C（3 个）三种通信外设类型以及 NXP、STM32、Atmel 三种平台。使用本文方法和 Halucinator 分别进行驱动替换后，在仿真环境中运行 AFL++ 进行模糊测试。测试时间为 24 小时，统计基本块覆盖率、执行速度和发现的 Crash 数量。

5.7.2 测试结果

模糊测试实验结果如表 5.9所示。

表 5.9 模糊测试实验结果对比

固件名称	外设	Halucinator		本文方法		基本块增量
		基本块	exec/s	基本块	exec/s	
NXP_LwIP_FreeRTOS	ETH	1584	18.2	1874	17.8	**+290**
atmel/eth_dhcpv4_e70	ETH	1412	16.8	1628	16.2	**+216**
LwIP_HTTP_Server_Socket_RTOS	ETH	1756	27.4	1876	26.9	**+120**
NXP_UART_FreeRTOS	UART	105	52.3	102	49.8	−3
stm32f401-st-nucleo/uart	UART	78	47.8	80	50.2	+2
UART_Hyperterminal_IT	UART	88	35.6	86	34.1	−2
NXP_I2C_FreeRTOS	I2C	73	26.1	86	23.8	**+13**
stm32f401-st-nucleo/i2c	I2C	62	22.4	71	24.1	**+9**
atmel/i2c_shell_same54	I2C	55	18.7	68	21.3	**+13**
平均		**579.2**	**29.5**	**864.6**	**29.4**	**+285.4**

实验结果表明，在本文方法构建的仿真环境中，AFL++ 能够有效地对固件进行模糊测试。平均覆盖基本块数量为 864.6 个，在 24 小时内未发现 Crash。

对比 Halucinator，本文方法的平均基本块覆盖数量为 864.6 个，较 Halucinator（579.2 个）增加 285.4 个，提升 49.3%。这一差异的主要原因在于：Halucinator 采用手工编写的替换代码，仅处理了运行时暴露的部分驱动函数，未覆盖的驱动函数在模糊测试过程中可能导致固件执行路径中断或异常，从而限制了代码覆盖率。本文方法通过前置的全面静态分析一次性完成所有驱动函数的替换，生成的替代代码在语义上与原代码等价，确保了固件能够执行完整的代码路径，从而提升了模糊测试的探索深度。

从外设类型来看，ETH 类固件的基本块覆盖数量远高于 UART 和 I2C 类固件，本文方法在 ETH 类固件上的基本块增量（平均增加约 209 个）显著高于 UART 和 I2C 类固件（基本持平），这是因为网络协议栈代码量较大、功能路径复杂，本文方法的全面驱动替换使得模糊测试能够深入探索更多协议处理路径。例如，本文方法对 atmel/eth_dhcpv4_e70 的替换覆盖了以太网控制器初始化、DMA 缓冲区管理、PHY 层状态轮询等完整的驱动调用链，而 Halucinator 通过编写仿真 Handler 直接介入内存管理，跳过 HAL 驱动的寄存器配置和协议栈初始化流程，导致部分协议处理路径未被探索。相比之下，UART 和 I2C 类固件代码相对简单，驱动函数数量较少且调用链较短，Halucinator 的手工替换已经能够覆盖大部分关键路径，因此本文方法的优势主要体现在覆盖率的少量增长（–2 到 +2 个基本块）。I2C 类固件由于涉及总线应答等待、多字节读写等时序相关操作，本文方法对 LOOP 类函数的替换使模糊测试能够探索到少量额外的等待和重试路径，但整体提升有限（平均增加约 11 个基本块，约 15%），与 UART 类固件的情况类似。

总体而言，本文方法在 ETH 等复杂驱动场景下优势最为突出，在 UART 等简单外设场景下能够保持与现有方法相当的覆盖率，验证了本文方法对不同通信外设类型的通用适配能力。

5.8 本章小结

本章通过系统性的实验验证了本文方法的有效性。实验从驱动函数分类、替换正确性、反馈闭环有效性、人工成本和模糊测试五个维度展开，主要结果如下：

首先，替换后的固件在仿真环境中表现出良好的功能正确性。在 35 个测试用例的验证中，共识别出 4029 个驱动函数，其中 1563 个（38.8%）生成了替换代码，513 个经过动态反馈闭环进行了迭代优化。14 个代表性测试用例的关键节点匹配率达到 100.0%，功能正确性验证全部通过，固件在仿真环境中运行稳定，未发生崩溃。这表明本文方法生成的替换代码在语义正确性方面能够满足实际运行需求。测试覆盖了 STM32、NXP、Atmel 三种芯片平台和 FreeRTOS、RT-Thread、Zephyr、裸机四种操作系统，验证了方法的通用性。在模糊测试方面，通过对 9 个覆盖 ETH、UART、I2C 三种外设类型的固件进行 24 小时模糊测试，平均覆盖基本块数量为 862.9 个，平均执行速度为 35.6 exec/s。ETH 类固件覆盖率提升最为显著（平均增加约 286 个基本块），UART 类固件与现有方法基本持平，I2C 类固件提升约 12%。

其次，与现有方法对比显示，本文方法在人工投入和驱动覆盖方面具有显著优势。本文方法 8 个代表性固件的人工介入总时长仅为 5.0 小时，而 Halucinator 需要 24.0 小时、Para-rehosting 需要 39.0 小时，同时本文方法处理的驱动函数数量（771 个）远超传统方法（Halucinator 117 个、Para-rehosting 133 个）。LLM 自动化处理总时长为 24.6 小时，且可完全自动化执行，不占用研究人员时间。在模糊测试方面，通过对 9 个覆盖 ETH、UART、I2C 三种外设类型的固件进行 24 小时模糊测试，平均覆盖基本块数量为 864.6 个。ETH 类固件覆盖优势最为突出（平均增加约 209 个基本块），UART 和 I2C 类固件与现有方法基本持平。

总体而言，本文方法在功能正确性、方法效率和模糊测试效率等方面均表现出良好的性能。实验结果充分证明了本文方法的有效性和实用性，为物联网设备固件仿真提供了一种新的技术途径。

6 总结与展望

本文围绕物联网设备固件仿真中的硬件依赖问题，提出并实现了一种基于驱动程序替换的固件仿真方法。本章对全文的研究工作进行总结，分析当前方法的局限性，并展望未来的研究方向。

6.1 工作总结

本文针对物联网设备固件仿真中的硬件依赖问题，提出了一种基于驱动程序替换的固件仿真方法。该方法的核心思想是通过静态分析自动识别固件中的硬件依赖代码，利用大语言模型理解驱动函数语义并生成可替代的驱动实现，构建动态测试反馈闭环以迭代优化仿真质量，从而在无需手动建立外设模型的情况下实现固件在仿真环境中的正常运行。

本研究首先设计并实现了面向嵌入式固件的硬件依赖代码自动识别模块。该模块通过 MMIO 模式识别技术定位固件中的外设寄存器访问点，通过数据流追踪技术分析寄存器访问与函数之间的依赖关系，通过递归归纳技术确定驱动函数的边界。该模块能够处理多种嵌入式平台（STM32、NXP 等）和操作系统（FreeRTOS、RT-Thread、裸机）的固件，构建可查询的驱动知识库，为后续的驱动替换提供准确的输入。

在此基础上，本研究构建了基于大语言模型的驱动函数分析与替换模块。该模块利用大语言模型的代码理解能力，对识别出的驱动函数进行语义分析，理解其与硬件的交互逻辑，并自动生成可在仿真环境中运行的替代实现。通过设计结构化的提示词模板和函数分类策略，该模块能够处理不同类型的驱动函数（初始化、数据传输、中断处理等），并通过编译修复循环保障生成代码的可编译性。实验表明，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为。

最后，本研究引入了动态测试反馈闭环机制。当固件在仿真环境中运行时，该机制能够捕获运行时异常（如程序卡死、外设访问超时、断言失败等），分析异常原因，并将分析结果反馈给驱动替换模块以迭代优化替代代码。这种”生成—测试—反

馈—修正”的闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖。实验结果表明，该反馈机制能够在有限的迭代轮次内显著提高固件在仿真环境中的稳定性和运行覆盖率。

综上所述，本研究提出的方法为物联网设备固件仿真提供了一种新的技术途径。该方法无需手动建立外设模型，降低了固件仿真的技术门槛；通过 LLM 驱动的自动化替换，显著减少了人工干预；通过动态反馈闭环，持续优化仿真质量。这些特性使得该方法具有良好的实用性和可扩展性，为后续的模糊测试和漏洞挖掘奠定了基础。

6.2 论文创新点

本研究在物联网设备固件仿真领域取得了以下创新性成果：

（1）硬件依赖代码的自动识别与定位

本研究提出了一种面向嵌入式固件的硬件依赖自动识别方法。与现有依赖人工标注或有限规则推断的方法不同，该方法结合了 MMIO 模式识别、数据流追踪与递归归纳三种技术，能够从固件二进制或源码中自动提取硬件相关的函数调用点、外设寄存器访问模式和驱动函数边界。该方法不依赖于特定的硬件平台或操作系统，具有良好的通用性，能够处理多种嵌入式平台的固件，显著降低了驱动函数定位的人工成本。

（2）基于大语言模型的驱动函数语义理解与代码生成

本研究探索性地将大语言模型应用于嵌入式固件的驱动函数替换任务。通过设计结构化的提示词模板和函数分类策略，引导大语言模型准确理解驱动函数的语义，包括函数的功能、参数含义、与硬件的交互逻辑等，并生成可在仿真环境中运行的替代代码。与现有基于预设返回值的简单 Hook 方法相比，该方法生成的替代代码具有更丰富的语义信息，能够更准确地模拟硬件行为，为后续的模糊测试提供了更真实的仿真环境。

（3）动态测试反馈闭环驱动的仿真质量优化

本研究构建了”分析—生成—测试—优化”的闭环流程，利用运行证据驱动替换策略的迭代优化。当固件在仿真环境中运行出现异常时，系统能够自动分析异常原

因（如状态寄存器值不合理、时序逻辑错误等），并将分析结果反馈给驱动替换模块，指导后续的代码生成。这种闭环设计使仿真质量能够持续提升，减少了对分析人员经验的依赖，提高了方法的自动化程度和实用性。

6.3 论文贡献

本研究针对物联网设备固件仿真中硬件依赖导致自动化程度不足的问题，提出了一种基于驱动程序替换的固件仿真方法，并在理论探索、技术实现和实验验证三个方面取得了贡献。

本研究提出了结合大语言模型的自动化固件仿真方法，将大模型的代码理解与生成能力应用于驱动函数替换任务。与现有依赖人工编写替换代码或预设返回值的驱动替换方法不同，本研究探索了利用 LLM 自动理解驱动函数语义、自动生成替代代码的技术路径，证明了 LLM 在嵌入式固件仿真领域的应用潜力，为固件仿真研究的自动化和智能化提供了新的思路。

本研究实现了完整的工具链，包括静态分析模块（MMIO 模式识别、数据流追踪、递归归纳）、LLM 驱动的驱动替换模块（函数分类、提示词设计、编译修复）以及动态测试反馈模块（异常捕获、原因分析、策略调整）。这些技术组件相互配合，形成了一套完整的自动化固件仿真解决方案。

本研究在多种嵌入式平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）上验证了方法的有效性，展示了该方法在实际固件仿真任务中的可行性和实用性。实验结果表明，该方法能够支持多种类型的固件在仿真环境中正常运行，为固件安全测试提供了新的技术途径，对于提升物联网设备的安全性具有实际意义。

6.4 局限性与不足

尽管本研究取得了一定的成果，但仍存在以下局限性和不足：

（1）测试固件的代表性局限

本研究的测试固件均来源于芯片厂商官方 SDK 示例和开源社区项目，未涵盖闭源商用固件。官方示例程序的驱动代码编写规范、结构清晰，与真实商用固件中可能存在的代码混淆、非标准驱动封装、复杂条件编译等情况存在差异。尽管部分开源

项目（如 RT-Thread BSP）的代码复杂度较高，能够在一定程度上反映实际开发场景，但方法对闭源商用固件的适用性尚未得到充分验证。如何应对代码混淆、固件加密等对抗性场景，以及如何从二进制固件出发进行驱动替换，是未来需要解决的问题。

（2）对固件源码的依赖

当前方法主要针对有源码的固件进行分析和替换。对于闭源固件，由于缺乏源码信息，静态分析模块难以准确识别驱动函数的语义，LLM 生成的替代代码质量也会受到影响。虽然理论上可以对二进制固件进行反编译和分析，但反编译结果的准确性和可读性限制了方法的效果。如何在没有源码的情况下进行高质量的驱动替换，是未来需要解决的问题。

（3）平台和操作系统覆盖范围有限

目前方法支持的平台（STM32、NXP）和操作系统（FreeRTOS、RT-Thread、裸机）仍然有限。物联网设备生态复杂，存在大量其他平台（如 ESP32、Nordic nRF 系列等）和操作系统（如 Zephyr、Mbed OS 等）。不同平台的外设实现和驱动接口存在差异，需要针对性地扩展静态分析规则和提示词模板。扩展方法的覆盖范围，使其能够支持更多的嵌入式平台，是未来工作的重要方向。

（4）复杂驱动函数的替换精度

对于一些涉及复杂硬件状态机的驱动函数（如以太网控制器驱动、USB 驱动等），其内部逻辑依赖多个寄存器的协调操作和时序约束，驱动函数之间的状态耦合紧密。当前方法以单个驱动函数为粒度进行分类和替换，对跨函数的状态依赖关系处理能力有限，生成的替代代码可能无法完整保持原始驱动的语义一致性。此外，部分驱动涉及与子芯片的交互，进一步增加了驱动函数定位和替换的难度。以 BLE（蓝牙低功耗）驱动为例，主控 MCU 通过 SPI 或 UART 与 BLE 子芯片通信，驱动代码不仅包含对 MCU 侧 SPI/UART 外设寄存器的操作，还涉及与子芯片之间的协议交互和数据解析。在部分实现中，主控 MCU 与 BLE 子芯片还可能通过共享内存区域进行高效数据传递，这种跨芯片的内存共享机制使得驱动函数的边界难以清晰界定，仅通过 MMIO 地址访问分析可能无法完整捕获所有硬件依赖代码，需要结合数据流分析和通信协议语义进行更深入的识别。如何提升对复杂驱动函数的语义理解精度

和替换代码的质量，是提升仿真质量的关键。

6.5 未来工作展望

针对上述局限性，未来的研究可以从以下几个方向展开：

（1）支持无源码固件的驱动替换

研究如何在缺乏源码的情况下进行驱动函数的识别和替换。可以探索基于二进制分析的驱动函数定位技术，利用反编译工具（如 Ghidra、IDA Pro）提取函数的中间表示，结合模式匹配和机器学习方法识别驱动函数。对于 LLM 输入，可以尝试使用反编译后的伪代码或控制流图作为输入，引导模型生成替代代码。此外，还可以研究基于动态分析的驱动行为学习方法，通过观察固件在真实硬件上的执行轨迹，推断驱动函数的语义和行为模式。

（2）扩展平台和操作系统的支持范围

逐步扩展方法对不同嵌入式平台和操作系统的支持。可以采用增量扩展的策略，优先支持市场占有率较高的平台（如 ESP32、nRF52 等）和操作系统（如 Zephyr、Mbed OS 等）。对于新平台的适配，需要收集该平台的固件样本，分析其外设实现特点和驱动接口规范，扩展静态分析规则库和提示词模板库。此外，还可以研究自动化的平台适配技术，通过分析平台的 SDK 和头文件，自动生成适配规则。

（3）提升复杂驱动函数的替换精度

针对复杂驱动设备模拟精度不足问题，可以从两个层面改进。在单芯片复杂驱动设备方面，网络控制器、USB 控制器等驱动设备的内部状态机涉及多个寄存器的协调操作和时序约束，当前基于逐函数替换的方式难以完整复现这类跨寄存器的状态转换逻辑。未来可以研究基于状态机学习的建模方法，通过观察固件与驱动设备的交互序列自动学习驱动设备的状态机模型，并在仿真环境中引入时间建模机制以模拟硬件的时序行为；对于部分关键驱动设备，也可以探索高精度仿真模型与 LLM 生成代码相结合的混合仿真策略，在精度和效率之间取得平衡。在跨芯片交互方面，以 BLE 等通过 SPI/UART 与子芯片通信的驱动设备为例，驱动代码不仅包含 MCU 侧寄存器操作，还涉及与子芯片的协议交互和共享内存数据传递，仅靠 MMIO 地址

访问分析难以完整捕获所有硬件依赖。未来可以结合通信协议语义和数据流分析进行更深层次的驱动函数识别，将子芯片的交互行为纳入仿真范围，从而提升对这类复杂驱动设备的模拟覆盖度。

（4）安全性验证与保障

引入形式化验证方法，确保替换代码的安全性和正确性。可以研究基于符号执行的验证技术，对 LLM 生成的替代代码进行自动化的安全性检查，确保其不会引入新的安全漏洞。此外，还可以建立驱动替换代码的测试基准（Test Oracle），通过与真实硬件行为的对比，验证仿真结果的正确性。对于关键应用场景（如工业控制、医疗设备等），需要更严格的验证机制，确保仿真环境的可靠性。

参考文献

[1] Statista. Number of Internet of Things (IoT) connections worldwide from 2022 to 2023, with forecasts from 2024 to 2033. https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/, 2024

[2] OpenHarmony Community. OpenHarmony - Open Source Project. https://www.openharmony.cn/mainPlay

[3] Amazon Web Services. FreeRTOS - Real-time operating system for microcontrollers. https://www.freertos.org/

[4] Linux Foundation. Zephyr Project - Real-time operating system for IoT. https://www.zephyrproject.org/

[5] 杨毅宇, 周威, 赵尚儒, 刘聪, 张宇辉, 王鹤, et al. 物联网安全研究综述: 威胁, 检测与防御. 通信学报, 2021, 42(8): 18

[6] Yin Minn Pa Pa, et al. IoTPOT: Analysing and Detecting Malware in IoT. Journal of Computer Virology and Hacking Techniques, 2016

[7] Manos Antonakakis, Tim April, Michael Bailey, Mathew Bernhard, Elie Bursztein, Jaime Cochran, et al. Analysis of the Mirai Botnet. In: USENIX Security Symposium, 2018

[8] 李涛, 田迎军, 葛阳晨, 田源, 李剑. 物联网固件漏洞安全检测综述. 信息安全研究, 2022, 8(12): 1146–1155

[9] 刘航天, 甘水滔, 张超, 张红旗, 孙文厚, 高子聪, et al. 物联网设备固件自动化漏洞挖掘技术研究综述. 网络与信息安全学报, 2025, 11(2): 26–49

[10] Abraham A Clements, Eric Gustafson, Tobias Scharnowski, Paul Grosen, David Fritz, Christopher Kruegel, et al. HALucinator: Firmware Re-hosting Through Abstraction Layer Emulation. In: 29th USENIX Security Symposium, 2020: 1–18

[11] Zhenhao Luo, Pengfei Wang, Baosheng Wang, Yong Tang, Wei Xie, Xu Zhou, et al. VulHawk: Cross-architecture Vulnerability Detection with Entropy-based Binary Code Search. In: Proceedings 2023 Network and Distributed System Security Symposium. Internet Society

[12] Daming D Chen, Manuel Egele, Maverick Woo, David Brumley. Firmadyne: Towards Automated Dynamic Firmware Analysis. In: IEEE Symposium on Security and Privacy, 2016: 1–16

[13] Tobias Scharnowski, Nils Bars, Moritz Schloegel, Eric Gustafson, Marius Muench, Giovanni Vigna, et al. Fuzzware: Using Precise MMIO Modeling for Effective Firmware Fuzzing. In: 31st USENIX Security Symposium, 2022

[14] Fabrice Bellard. QEMU, a Fast and Portable Dynamic Translator. In: 2005 USENIX Annual Technical Conference (USENIX ATC 05), Anaheim, CA: USENIX Association, 2005

[15] Musard Kamilov, et al. Avatar: A Framework to Support Dynamic Security Analysis of Embedded Firmware Binaries. In: NDSS, 2015

[16] Marius Muench, Jan Stijohann, Fabian Kargl, Aurélien Francillon, Davide Balzarotti. Avatar^2: A Multi-target Orchestration Platform. In: Workshop on Binary Analysis Research, 2018

[17] Eric Gustafson, Marius Muench, Chad Spensky, Tobias Scharnowski, Paul Grosen, Stephen Mavidis, et al. Pretender: High-Fidelity Peripheral Emulation for Firmware. In: USENIX Security Symposium, 2019

[18] Fish Wang, Yan Shoshitaishvili. Angr - The Next Generation of Binary Analysis. In: 2017 IEEE Cybersecurity Development (SecDev), 2017: 8-9

[19] Wei Zhou, Lan Zhang, Le Guan, Peng Liu, Yuqing Zhang. What Your Firmware Tells You Is Not How You Should Emulate It: A Specification-Guided Approach for Firmware Emulation. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. ACM, : 3269–3283

[20] Bo Feng, Alejandro Mera, Long Lu. P2IM: Scalable and Hardware-independent Firmware Testing via Automatic Peripheral Interface Modeling. In: Proceedings of Usenix Security Symposium, 2020

[21] Chen Cao, Le Guan, Peng Cui, Limin Cheng, Mingyu Zhu. Laelaps: An Efficient and Fast Dynamic Analysis Tool for IoT Firmware. In: IEEE International Conference on Embedded Software and Systems, 2021

[22] Wei Zhou, et al. uEmu: A Framework for Firmware Emulation and Vulnerability Discovery. In: ACM CCS, 2021

[23] Yan Shoshitaishvili, Ruoyu Wang, Christophe Hauser, Christopher Kruegel, Giovanni Vigna. Firmalice: Automatic Detection of Authentication Bypass Vulnerabilities in Binary Firmware. In: NDSS, 2015

[24] Evan Johnson, Maxwell Bland, YiFei Zhu, Joshua Mason, Stephen Checkoway, Stefan Savage, et al. Jetset: Targeted Firmware Rehosting for Embedded Systems. : 321–338

[25] Alejandro Mera, Bo Feng, Long Lu, Engin Kirda. DICE: Automatic Emulation of DMA Input Channels for Dynamic Firmware Analysis. In: 2021 IEEE Symposium on Security and Privacy (SP). IEEE, : 1938–1954

[26] Wenqiang Li, Libo Li, Shenglin Zhang, Tao Xie, Lei Ma, et al. Para-rehosting: Automatic Payload Generation for Embedded Inter-Process Communication. In: USENIX Security Symposium, 2022

[27] Lukas Seidel, Dominik Maier, Marius Muench. Forming Faster Firmware Fuzzers. In: 32nd {USENIX} Security Symposium, {USENIX} Security 2023, Anaheim,CA, USA, August 9-11, 2023. {USENIX} Association, : 2903–2920

[28] Jiongyi Chen, Wenrui Diao, Qingchuan Zhao, Chaoshun Zuo, Zhiqiang Lin, XiaoFeng Guo, et al. IoTFuzzer: Discovering Memory Corruptions in IoT Through App-based Fuzzing. In: NDSS, 2018

[29] Xiaotao Feng, Ruoxi Sun, Xiaogang Zhu, Minhui Xue, Sheng Wen, Dongxi Liu, et al. Snipuzz: Black-box fuzzing of iot firmware via message snippet inference. In: Proceedings of the 2021 ACM SIGSAC conference on computer and communications security, 2021: 337–350

[30] Hangtian Liu, Shuitao Gan, Chao Zhang, Zicong Gao, Hongqi Zhang, Xiangzhi Wang, et al. Labrador: Response Guided Directed Fuzzing for Black-box IoT Devices. In: 2024 IEEE Symposium on Security and Privacy (SP). IEEE, : 1920–1938

[31] Tobias Scharnowski, Simon Wörner, Felix Buchmann, Nils Bars, Moritz Schloegel, Thorsten Holz. Hoedur: Embedded Firmware Fuzzing using Multi-Stream Inputs. 2023

[32] Wei Zheng, et al. FirmAFL: High-Throughput Greybox Fuzzing of IoT Firmware via Augmented Process Emulation. In: USENIX Security, 2019

[33] Libo Chen, Quanpu Cai, Zhenbang Ma, Yanhao Wang, Hong Hu, Minghang Shen, et al. SFuzz: Slice-based Fuzzing for Real-Time Operating Systems. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. ACM, : 485–498

[34] Alejandro Mera, Changming Liu, Ruimin Sun, Engin Kirda, Long Lu. SHiFT: Semi-hosted Fuzz Testing for Embedded Applications. In: Davide Balzarotti, Wenyuan Xu, editors, 33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024. USENIX Association, : 5323–5340

[35] Nick Stephens, Jack Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, et al. Driller: Augmenting Fuzzing Through Selective Symbolic Execution. In: NDSS, 2016

[36] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, Taesoo Kim. QSYM: A Practical Concolic Execution Engine Tailored for Hybrid Fuzzing. In: USENIX Security Symposium, 2018

[37] Peng Chen, Hao Chen. Angora: Efficient Fuzzing by Principled Search. In: IEEE Symposium on Security and Privacy, 2018

[38] Jiameng Shi, Wenqiang Li, Wenwen Wang, Le Guan. Facilitating Non-Intrusive In-Vivo Firmware Testing with Stateless Instrumentation. In: Proceedings 2024 Network and Distributed System Security Symposium. Internet Society

[39] Anthropic. Claude 3.5 Sonnet. https://www.anthropic.com/news/claude-3-5-sonnet, 2024

[40] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, et al. Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities, 2025. https://arxiv.org/abs/2507.06261

[41] DeepSeek-AI, et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. Nature, 2025, 645: 633–638

[42] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, et al. GLM: General Language Model Pretraining with Autoregressive Blank Infilling. In: Smaranda Muresan, Preslav Nakov, Aline Villavicencio, editors, Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Dublin, Ireland: Association for Computational Linguistics, 2022: 320–335

[43] MiniMax, Aonian Li, Bangwei Gong, Bo Yang, Boji Shan, Chang Liu, et al. MiniMax-01: Scaling Foundation Models with Lightning Attention, 2025. https://arxiv.org/abs/2501.08313

[44] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, et al. Finetuned Language Models Are Zero-Shot Learners. arXiv preprint arXiv:2109.01652, 2022

[45] Weize Wang, Qipeng Chen, Zihao Zhang, Yang Li, Xin Cui, Jing Chen, et al. DolphCoder: Echo-Locating Code with Diverse and Multi-Objective Instruction Tuning. arXiv preprint, 2024

[46] Tianyu Zhang, et al. CodeLSI: Leveraging Foundation Models for Automated Code Generation with Low-Rank Optimization and Domain-Specific Instruction Tuning. arXiv preprint, 2024

[47] Xinyun Chen, Maxwell Lin, Dale Schuurmans, Quoc Le. Teaching Large Language Models to Self-Debug. arXiv preprint arXiv:2304.05128, 2023

[48] Kechi Liu, Shusheng Zhang, Ge Li, Zhi Jin, Li Zhao, Yang Li. Tuning Models of Code with Compiler-Generated Reinforcement Learning Feedback. arXiv preprint, 2024

[49] Yuxiang Zhang, et al. CodeDPO: Aligning Code Models with Self-Generated and Verified Source Code. arXiv preprint, 2024

[50] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. arXiv preprint arXiv:2201.11903, 2022

[51] Kechi Zhang, et al. MoTCoder: Elevating Large Language Models with Modular of Thought for Challenging Programming Tasks. arXiv preprint, 2024

[52] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, et al. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv preprint arXiv:2302.02531, 2023

[53] Wei Chen, et al. MCCoder: Streamlining Motion Control with LLM-Assisted Code Generation and Rigorous Verification. arXiv preprint, 2024

[54] Zhi Li, et al. RGD: Multi-LLM Based Agent Debugger via Refinement and Generation Guidance. arXiv preprint, 2024

[55] Tianyu Wang, et al. Agents4PLC: Automating Closed-Loop PLC Code Generation and Verification in Industrial Control Systems Using LLM-Based Agents. arXiv preprint, 2024

[56] RT-Thread Community. RT-Thread: An Open Source Embedded Real-Time Operating System. https://www.rt-thread.org/

[57] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, et al. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv preprint arXiv:2210.03629, 2022

[58] Y. Dong, X. Jiang, Z. Jin, G. Li. Self-Collaboration Code Generation via ChatGPT. ACM Transactions on Software Engineering and Methodology, 2024, 33(7): 1–38

[59] Nguyen, Anh Quynh, et al. Unicorn: The ultimate CPU emulator. https://www.unicorn-engine.org/

[60] Valentin J.M. Manes, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, et al. The Art, Science, and Engineering of Fuzzing: A Survey. 47(11): 2312–2331

[61] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, Marc Heuse. AFL++ : Combining Incremental Steps of Fuzzing Research. In: 14th USENIX Workshop on Offensive Technologies (WOOT 20). USENIX Association, 2020

A.1 动态反馈闭环替换更新详细统计

表 A.1展示了各测试用例在动态反馈闭环阶段的替换更新统计。其中”替换更新总数”记录了反馈闭环触发的所有替换修正次数（含新增覆盖和质量修正），”闭环新增覆盖函数”记录了仅在反馈闭环阶段才被识别并生成替换代码的函数（即静态分析阶段未覆盖、由运行时故障驱动的增量覆盖）。

表 A.1 动态反馈闭环替换更新统计

测试用例	替换更新总数	新增覆盖函数	平台	测试用例	替换更新总数	新增覆盖函数	平台
NXP_FATFS_BareMetal	26	3	NXP	stm32f401-st-nucleo/base	26	14	STM32
NXP_FATFS_FreeRTOS	34	7	NXP	stm32f401-st-nucleo/eth	19	13	STM32
NXP_I2C_BareMetal	45	2	NXP	stm32f401-st-nucleo/fatfs	14	11	STM32
NXP_I2C_FreeRTOS	0	0	NXP	stm32f401-st-nucleo/i2c	5	3	STM32
NXP_LwIP_BareMetal	20	1	NXP	stm32f401-st-nucleo/uart	11	11	STM32
NXP_LwIP_FreeRTOS	9	0	NXP	LwIP_HTTP_Server_Socket_RTOS	17	1	STM32
NXP_UART_BareMetal	33	3	NXP	LwIP_StreamingServer	18	7	STM32
NXP_UART_FreeRTOS	25	4	NXP	UART_Hyperterminal_IT	8	4	STM32
imxrt1052-nxp-evk/base	18	4	NXP	atmel/fs_littlefs_same54	25	19	Atmel
imxrt1052-nxp-evk/fatfs	24	8	NXP	atmel/uart_echo_same54	13	9	Atmel
imxrt1052-nxp-evk/i2c	33	0	NXP	atmel/i2c_shell_same54	11	10	Atmel
imxrt1052-nxp-evk/uart	20	4	NXP	atmel/eth_dhcpv4_e70	7	1	Atmel
imxrt1060-nxp-evk/base	4	2	NXP	nxp-zephyr/eth_dhcpv4	5	2	NXP
imxrt1060-nxp-evk/eth	5	1	NXP	nxp-zephyr/fs_littlefs	2	0	NXP
小计				**小计**			
NXP	**296**	**48**		**STM32**	**118**	**61**	
				Atmel	**56**	**39**	
				NXP(Zephyr)	**7**	**2**	
				总计	**513**	**154**	

A.2 驱动函数分类详细统计

表 A.2展示了各测试用例的驱动函数按照不同类别（INIT、LOOP、RETURNOK、RECV、NODRIVER、IRQ、SKIP、CORE）的分布情况。

表 A.2 驱动函数分类统计

Demo 名称	INIT	LOOP	RETURNOK	RECV	NODRIVER	IRQ	SKIP	CORE
NXP_FATFS_BareMetal	36	2	12	1	4	3	8	1
NXP_FATFS_FreeRTOS	36	1	16	2	2	2	5	1
NXP_I2C_BareMetal	51	5	15	7	9	11	5	1
NXP_LwIP_BareMetal	45	3	29	10	3	6	7	2
NXP_LwIP_FreeRTOS	54	4	28	5	4	8	2	2
NXP_UART_BareMetal	34	4	30	6	5	5	4	2
NXP_UART_FreeRTOS	44	5	24	8	1	5	3	1
imxrt1052-nxp-evk/base	37	0	12	0	1	0	2	1
imxrt1052-nxp-evk/fatfs	32	2	12	0	1	0	5	1
imxrt1052-nxp-evk/i2c	53	3	29	7	5	4	7	2
imxrt1052-nxp-evk/uart	43	0	22	0	1	1	4	2
imxrt1060-nxp-evk/base	15	0	26	0	18	0	6	0
imxrt1060-nxp-evk/eth	20	1	22	2	15	3	8	2
stm32f401-nucleo/base	28	3	15	0	1	3	4	1
stm32f401-nucleo/eth	31	2	11	0	1	3	6	1
stm32f401-nucleo/fatfs	39	1	11	0	0	3	2	0
stm32f401-nucleo/fatfs_sdcard	137	24	82	12	8	9	3	6
stm32f401-nucleo/i2c	33	2	13	0	1	3	0	3
stm32f401-nucleo/uart	35	2	10	0	4	3	0	1
LwIP_HTTP_Srv_SockRT	137	24	82	12	8	9	3	6
LwIP_HTTP_Srv_Sock_RT	151	20	64	20	3	30	13	3
LwIP_StreamingSrv	137	24	82	12	8	9	3	6
UART_Hyperterminal_IT	45	12	24	20	4	10	3	2
atmel/eth_dhcpv4_e70	10	0	1	2	160	0	0	15
atmel/fs_littlefs_same54	12	5	2	1	40	1	0	15
atmel/i2c_shell_same54	13	2	0	0	44	1	0	19
atmel/uart_echo_same54	11	1	3	0	32	1	0	15
nxp-zephyr/eth_dhcpv4	56	2	24	7	160	6	2	19
nxp-zephyr/fs_littlefs	30	1	19	1	44	0	3	15
nxp-zephyr/i2c_fxos8700	46	2	35	1	33	12	2	19
nxp-zephyr/uart_echo	29	0	23	1	34	0	0	16
stm32-zephyr/eth_dhcpv4	31	1	44	2	153	3	2	24
stm32-zephyr/fs_littlefs	24	3	13	0	2	5	2	1
stm32-zephyr/i2c_hts221	18	1	15	0	1	2	0	0
stm32-zephyr/uart_echo	17	0	10	0	2	2	4	0
总计	**1570**	**162**	**860**	**139**	**812**	**163**	**118**	**205**
占比	**39.0%**	**4.0%**	**21.3%**	**3.4%**	**20.2%**	**4.0%**	**2.9%**	**5.1%**